Ecole Centrale de Casablanca (ECC)

Introduction aux techniques d'Apprentissage Automatique

(Machine Learning)

Prof. Y. Lamrani Alaoui
lamrani@emi.ac.ma
EMI-Rabat

Centrale
CASABLANCA



Plan de cours

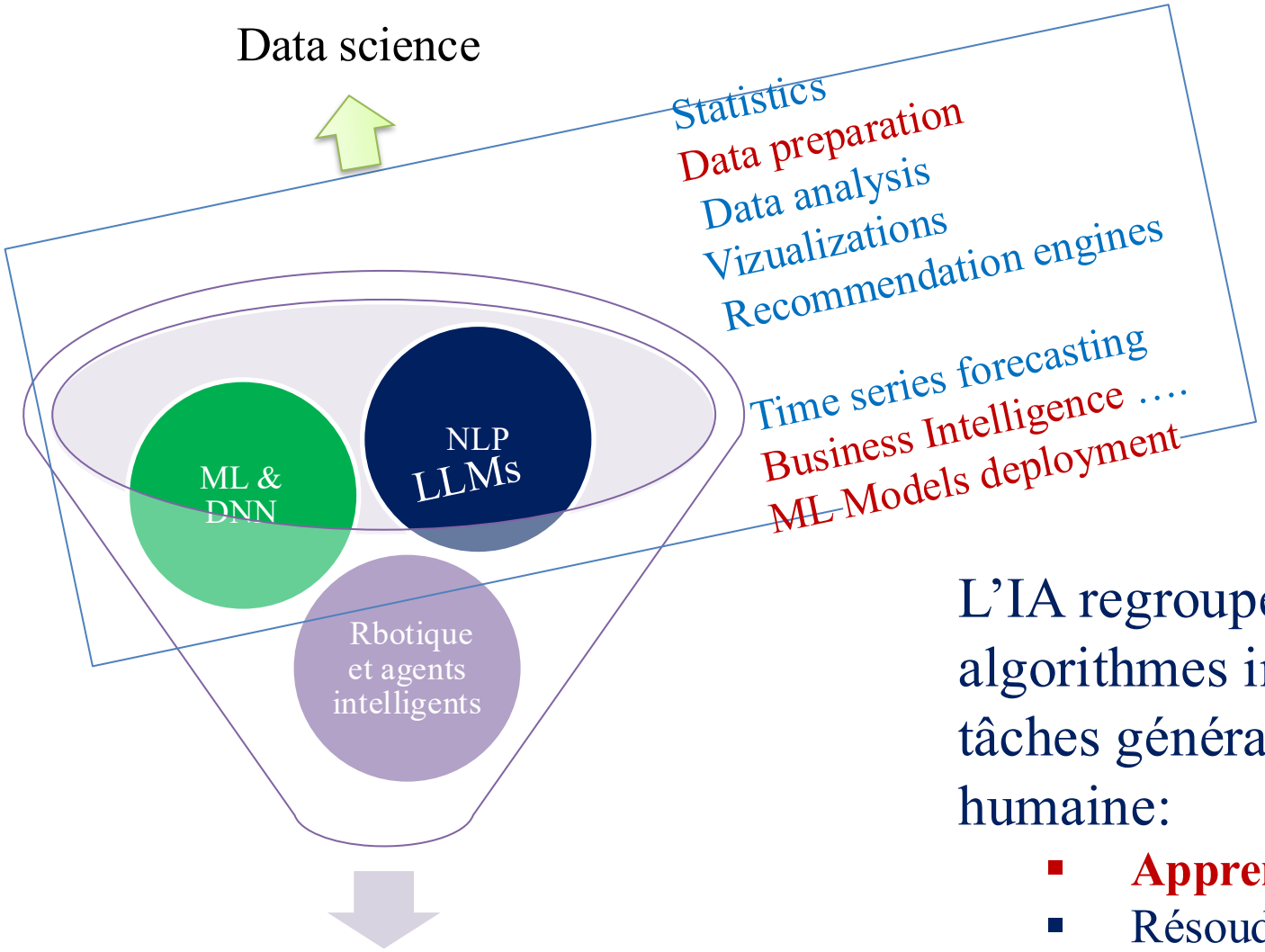
1. **Introduction et notions de base**
   - Panorama des paradigmes d'apprentissage
2. **Apprentissage supervisé**
   - Régression, dont régression régularisée (Ridge, Lasso, Elastic Net)
   - Classification binaire et multi-classes
   - Évaluation et optimisation des hyperparamètres
3. **Apprentissage non supervisé**
   - Réduction de dimension
   - Clustering : partition, hiérarchique, densité, modèles probabilistes
   - Détection d'anomalies
4. **Apprentissage semi-supervisé et par renforcement**
   - Hypothèses du semi-supervisé et pseudo-étiquetage
   - Renforcement : valeur, politique, acteur–critique, applications
5. **Traitement du langage naturel (NLP)**
   - Représentation du texte et tâches classiques du NLP
6. **Introduction à l'IA générative**
   - LLM : pré-entraînement, SFT, RLHF, prompt engineering
   - Systèmes RAG et IA agentique

Prérequis

- Programmation R et Python
- Statistiques descriptives et inférentielles
- Notions de base en calcul scientifique (Algèbre linéaire, Analyse, Optimisation,…)

Introduction et notions de base

Intelligence Artificielle et Data science



L'IA regroupe l'ensemble des techniques et algorithmes informatiques capables d'effectuer des tâches généralement associées à l'intelligence humaine:

- **Apprendre**
- Résoudre de problèmes
- Prendre une décision….

Robots et systèmes autonomes

La robotique: le design et la conception des machines capables d'interagir physiquement avec leur environnement grâce à des capteurs, actionneurs (actions) et des algorithmes d'IA (décision)

Exemples d'utilisation :

- Accompagner les personnes âgées ou avec une mobilité réduite
- La livraison des médicaments à des organes, tissues ou tumeurs cibles
- Enseigner et communiquer avec les enfants autistes



Fig. Un robot pour déplacer les patients fragiles d'un fauteuil roulant à un lit ou à une baignoire.

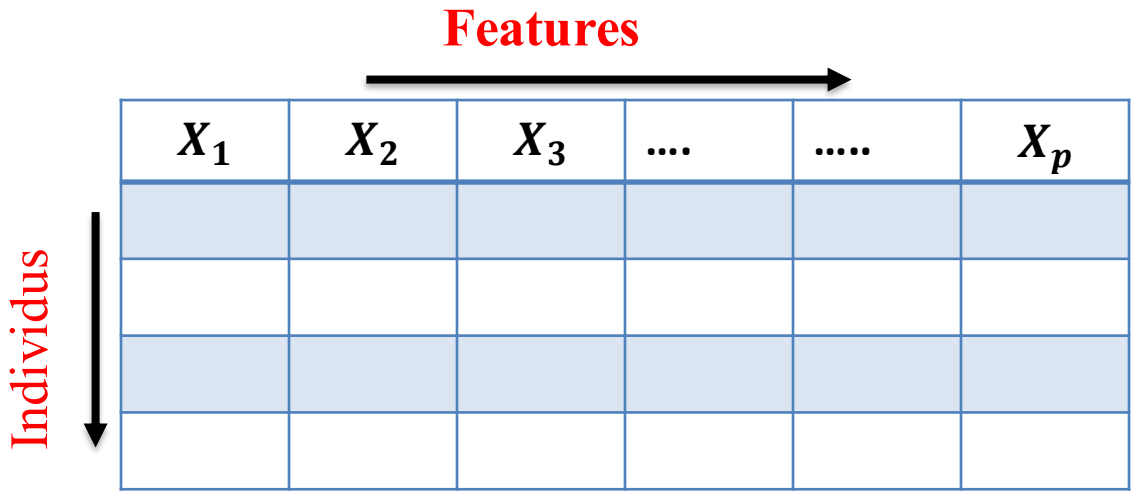
Traitement du langage naturel (NLP)

- L'objectif du **NLP** est de permettre aux ordinateurs de comprendre, interpréter et manipuler le langage humain (textuel ou vocal).
- Il combine **linguistique** et **apprentissage automatique** pour convertir le langage en représentations numériques exploitables par la machine.
- Il permet ainsi de lire un texte, de le traduire ou de le résumer, d'entendre un discours et de l'interpréter.
- **Applications** : traduction automatique, résumé un texte, générer un code informatique, reconnaissance vocale, analyse de sentiments, agents conversationnels,…



Machine Learning

Le Machine Learning, l'apprentissage automatique ou d'apprentissage statistique, consiste à créer des modèles à partir des données.

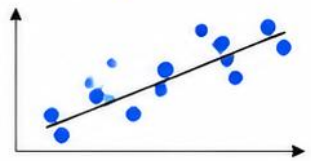


Forme de données

Panorama des paradigmes d'apprentissage

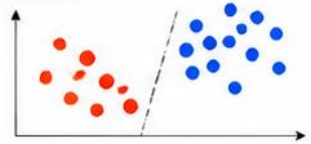
1 SUPERVISÉ

Régression

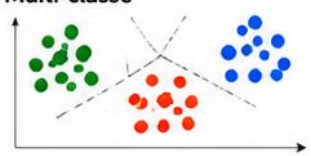


Classification

Binaire



Multi-classe

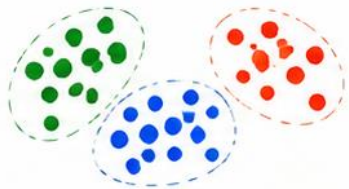


Multi-étiquettes

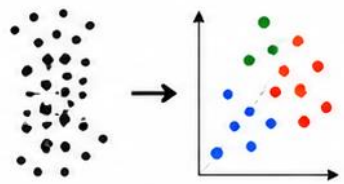
	✓	✓	✓
✓	✓	✓	✓

2 NON SUPERVISÉ

Clustering



Réduction de la dimension



Détection d'anomalie

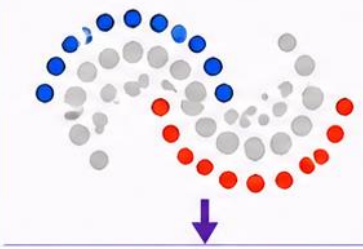


3 SEMI SUPERVISÉ

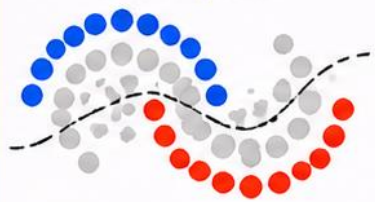
Peu de points étiquetés
+ beaucoup de points
non étiquetés

Classe 1 Classe 2
Non étiquetés

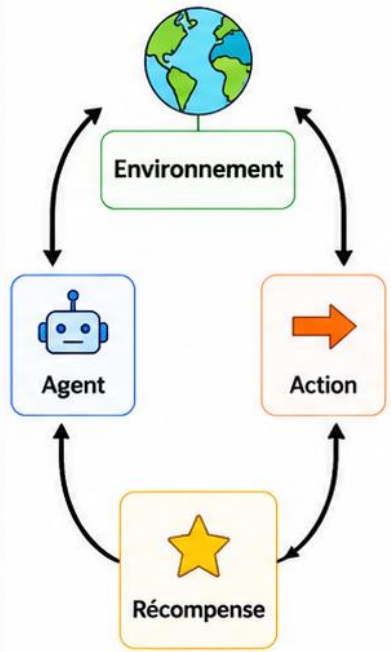
Données partielles



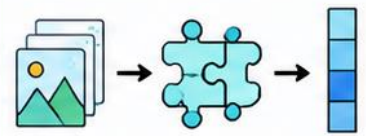
Prédiction grâce aux non étiquetés



4 PAR RENFORCEMENT



5 AUTO-SUPERVISÉ



Données brutes → Tâche prétexte → Représentations apprises

Apprentissage supervisé (supervised learning)

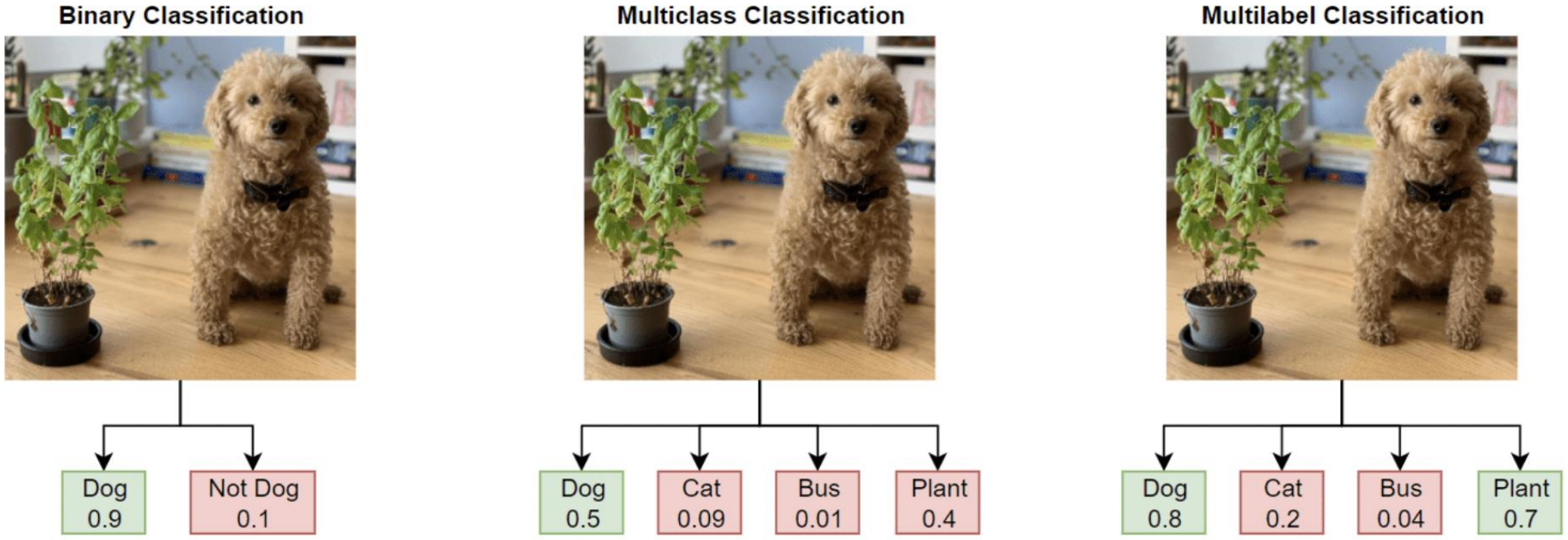
X_1	X_2	…	X_p	y

$$y = f(X_1, X_2, \ldots, X_p)$$

- *Les algorithmes supervisés extraient de la connaissance à partir d'un ensemble de données contenant des couples* ***entrées-sorties***
- *L'objectif est de modéliser la relation entre des variables explicatives (X) et une variable à expliquer (Y). Une fois le modèle est entrainé, il peut être utilisé pour prédire les sorties (Y) des nouvelles données non encore* ***labellisées.***

2 types d'apprentissage supervisé

Classification **:** *un modèle est dit de classification, si la variable à expliquer est discrete (variable catégorielle)*



Régression: un modèle est dit de régression, si la variable à expliquer est **continue** (quantitative)

Exemples de problèmes de classification

- **Classifier les emails**

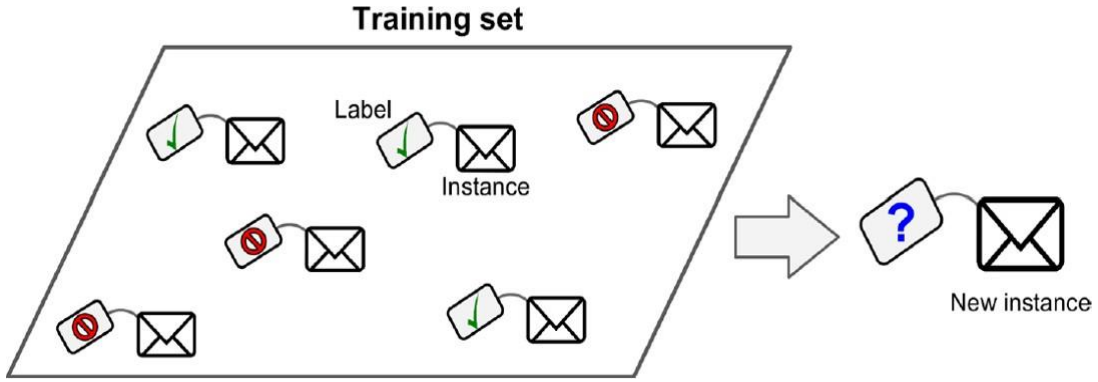


Figure: labeled training set



- **Identifier les activités frauduleuses par les transactions réalisées par les cartes de crédits**

Quelques algorithmes de Classification:

- Naïve bayes classification
- Support vector machines (SVM)
- Decision trees
- Random forest
- Logistic regression
- Linear discriminant analysis (LDA)
- Xgboost
- Gradient Boosting
- CatBoost
- K-Nearest Neighbors (KNN)

Exemple des problèmes de régression :

- **Prédire le prix d'une nouvelle voiture**



- **Prédire le prix d'une nouvelle Maison**



Price	Bedrooms	Bathrooms	Size
795000	3	3	2371
399000	4	3	2818
545000	4	3	3032
909000	4	4	3540
109900	3	1	1249
324900	3	3	1800
192900	4	2	1603
215000	3	2	1450
999000	4	3	3360
319000	3	2	1323

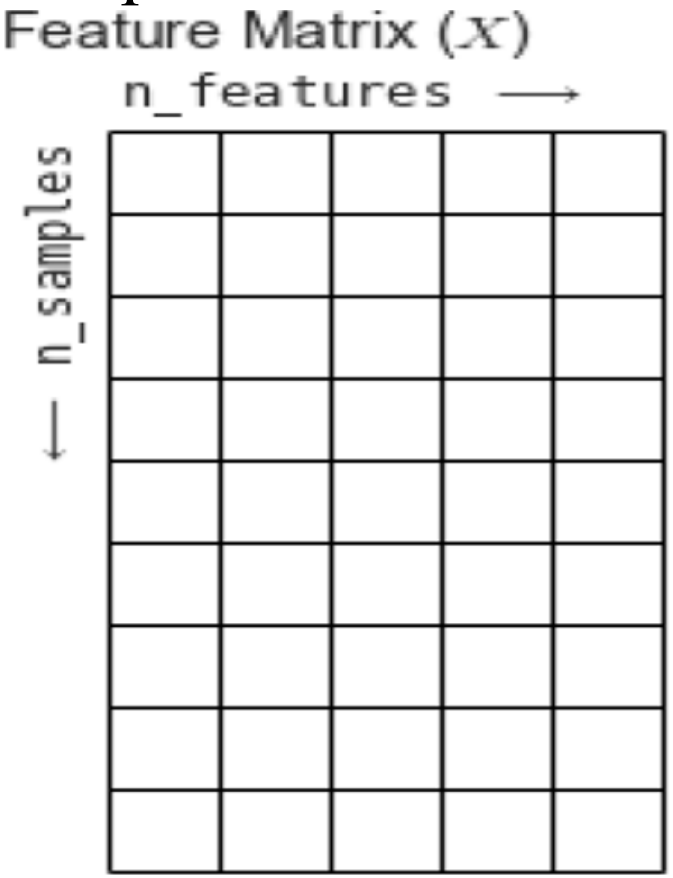
Table : House data set

Quelques algorithmes de régression :

- Linear Regression
- Ridge Regression
- LASSO Regression
- Elastic Net regression
- K-Nearest Neighbors (KNN)
- Polynomial Regression
- Bayesian regression
- Support vector machines
- Decision Trees
- Random forest Regression
- Lightgbm

Apprentissage non supervisé (Unsupervised learning)

- Les algorithmes non supervises n'intègrent pas la notion d'entrée-sortie. Toutes les données sont équivalentes.



Objectif

a) Réduire le nombre de variables (Réduction de dimensions)

b) Organiser les données similaires en groupes homogènes (**Clustering** ou **classification non supervisée**).

"Unsupervised learning is often described as :letting the dataset speak for itself."

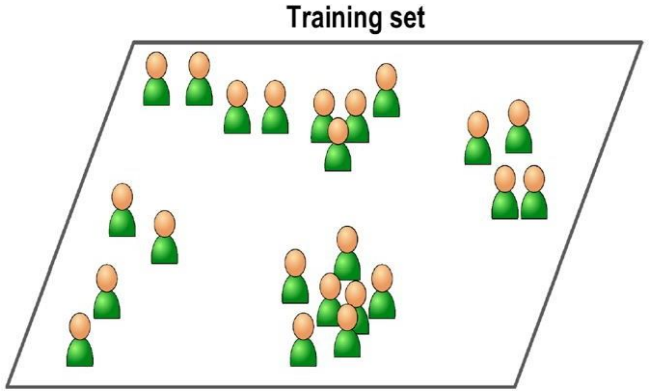


Figure: unlabeled training set

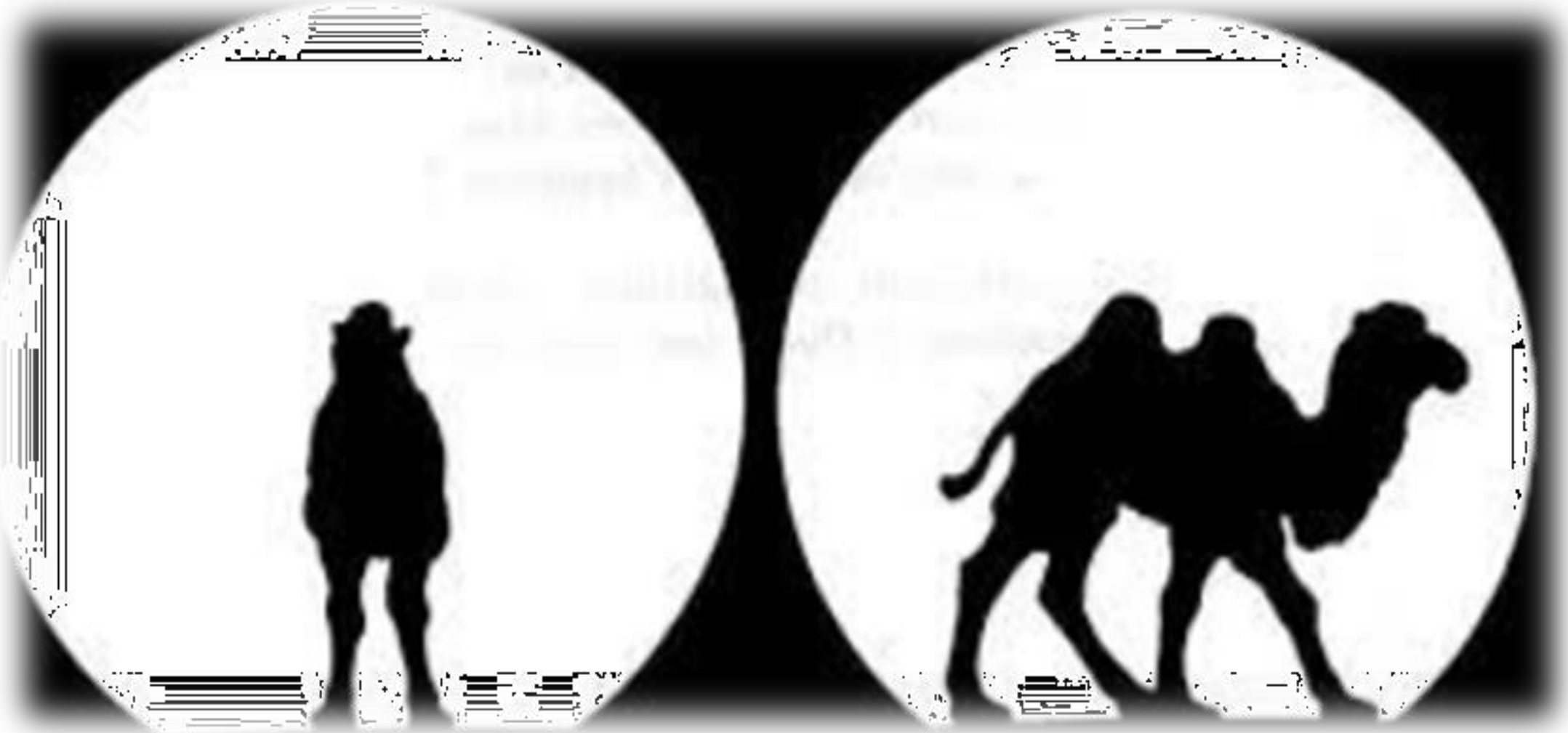
c) Détection d'anomalies: c'est le processus d'identification d'éléments, d'événements ou d'observations rares qui se distinguent significativement de la majorité des données.

Réduction de dimensions

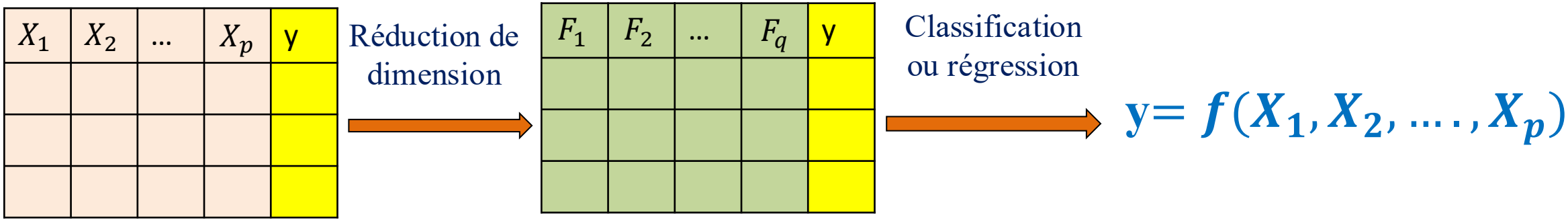
(Dimensionality Reduction)

- L'objectif est de chercher une représentation simplifiée de notre base de données:
  - Réduire le nombre de variable tout en minimisant la perte de l'information.
  - L'idée est de remplacer les variables fortement corrélées par des nouvelles variables indépendants (*feature extraction*)
  - Visualiser les relations importantes dans une base de données avec beaucoup des variables (dimensions)





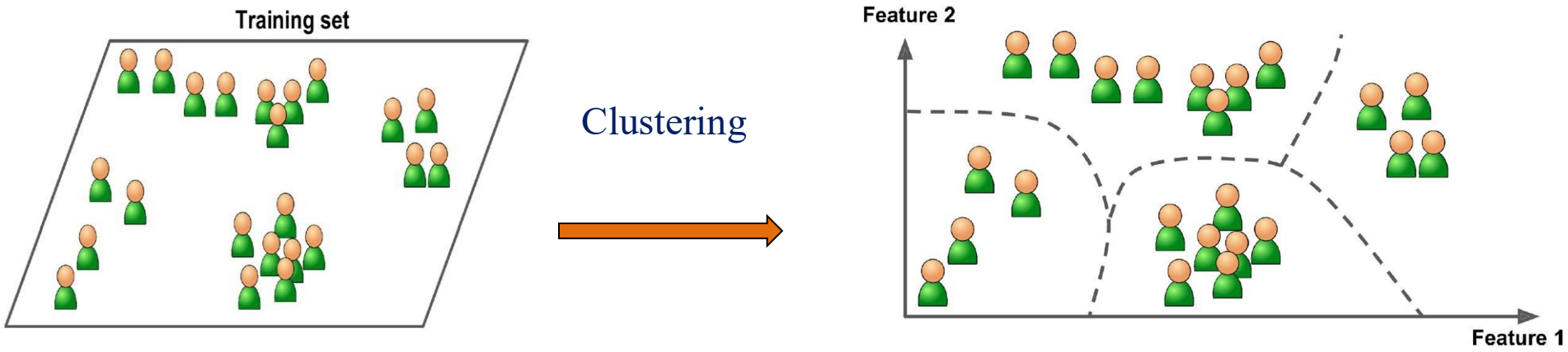
Réduction de dimension pour apprentissage supervisé



- Accélérer le processus d'apprentissage en diminuant le temps nécessaire pour estimer les paramètres.
- Réduire l'espace mémoire requis, ce qui est particulièrement utile lorsque l'on travaille avec de grandes quantités de données.
- Améliorer la performance de l'algorithme d'apprentissage en réduisant le risque de surajustement via élimination des variables redondantes ou non informatives

Clustering

❑ Regrouper les individus ayant des caractéristiques similaires.



Exemple:

- Regrouper les consommateurs ayant des préférences similaires.
- L'objectif peut être adapter un produit selon chaque groupe afin d'améliorer la satisfaction client

Détection d’anomalies

- La détection d'anomalies est le processus d'identification d'éléments, d'événements ou d'observations rares qui se distinguent significativement de la majorité des données.
- La détection d’anomalies est utilisée dans de nombreux secteurs d’activités:
  - Finance: détection de fraude, fausse déclaration d’impôts,…
  - Fabrication: identifier les défauts et les dysfonctionnements des équipements
  - Cybersécurité: détecter les activités inhabituelles des réseaux
  - Soins de santés: identifier les conditions anormales des patients (maladies,…)
  - Consommation d'énergie: surveiller la consommation d’énergie et identifier les anomalies d’utilisation

Example d'algorithmes d'apprentissage non supervisé

Réduction de dimensions:

- Analyse en composantes principales, ACP (Principal component analysis)
- Régression LASSO
- Auto-encodeurs (architecture deeplearning)

Clustering:

- K-means algorithm
- Hierarchical clustering algorithm (CAH,…)
- Density-based clustering: (DBSCAN,…)

Exemples d'algorithmes d'apprentissage non supervisé

Détection d'anomalies :

- Méthodes statistiques :
  - Visualisation (box plot)
  - Z-score
  - Tests statistiques (test de Grubbs)
  - Critère de Chauvenet

- Méthodes d'apprentissage automatique (ML) :
  - Non supervisé : DBSCAN, modèle de mélange gaussien (GMM), Isolation Forest, One-Class SVM, auto-encodeurs, Local Outlier Factor (LOF), etc.
  - Supervisé : SVM, Random Forest, etc.

Apprentissage semi-supervisé

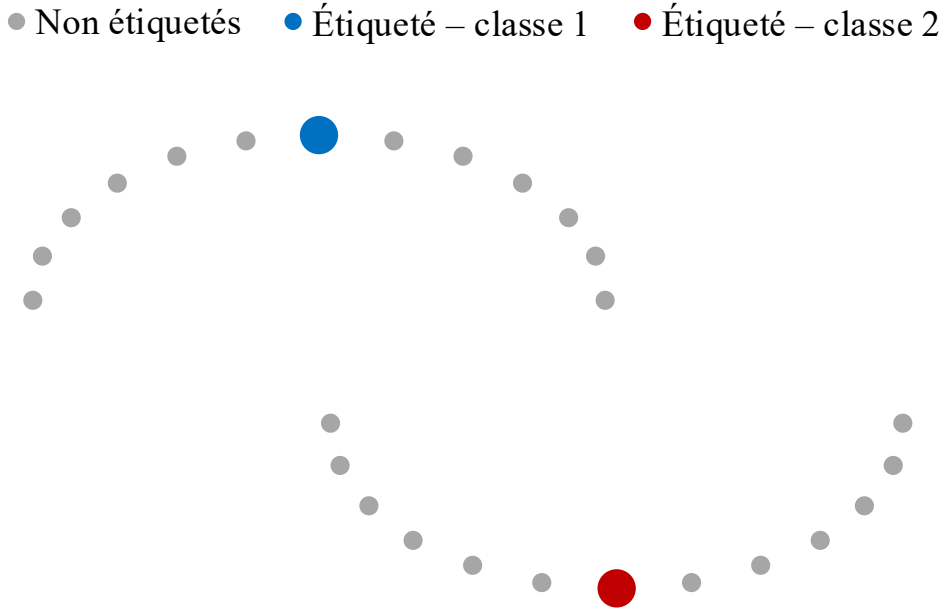
- On dispose d'un petit ensemble de données étiquetées $\mathcal{L} = \{x_i, y_i\}_{i=1}^{l}$ et d'un grand ensemble de données non étiquetées $\mathcal{U} = \{x_j\}_{j=1}^{u}$ avec $u \gg$ l.
- L'objectif est d'exploiter $\mathcal{U}$ pour améliorer le modèle appris sur l'ensemble étiqueté.

Deux hypothèses fondamentales :

1. Régularité (lissage) : si $x_i \sim x_j \rightarrow y_i = y_j$. Propriété locale : deux points voisins dans l'espace des caractéristiques reçoivent des étiquettes semblables.
2. Cluster : les points d'un même groupe dense partagent la même étiquette, même s'ils sont éloignés, dès lors qu'ils sont reliés par une région de forte densité. Propriété globale : la frontière de décision passe par les zones de faible densité.

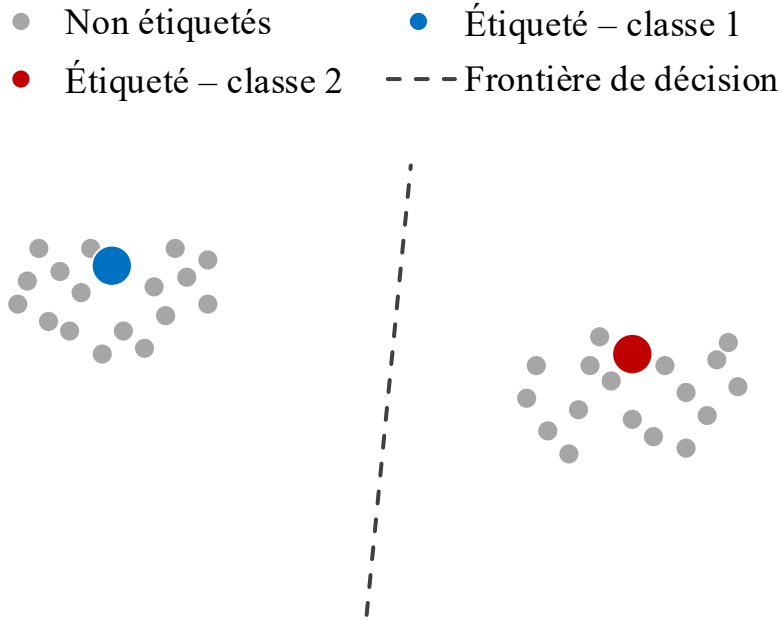
Apprentissage semi-supervisé

1. Régularité (lissage) – local



Propriété locale : les points proches reçoivent la même étiquette. L'étiquette se propage de voisin en voisin le long de la structure, même si les classes ne sont pas linéairement séparables.

2. Cluster / faible densité – global



Propriété globale : la frontière de décision est repoussée dans la zone de faible densité entre les deux groupes. Tous les points d'un même groupe dense héritent de la même étiquette.

Apprentissage semi-supervisé

Self-training approche:

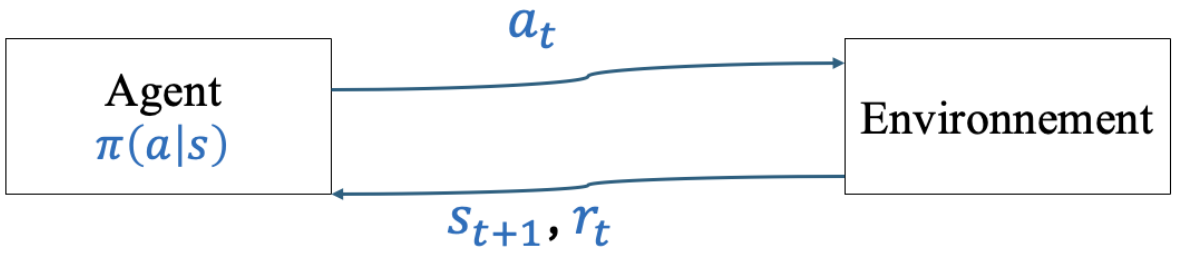
Entrainer un modèle sur les données étiquetées puis sélectionner les prédictions les plus confiantes (celle avec probabilités élevées) et les ajouter (pseudo-étiquettes) à l'ensemble d'entrainement pour réentraîner le modèle sur l'ensemble enrichi.

Remarque:

Si le modèle initial fait des erreurs systématiques sur certains points et les pseudo-étiquette avec confiance (à tort), ces erreurs se renforcent à chaque itération

Apprentissage par renforcement

- Un agent interagit avec un environnement, à chaque pas il observe un état s_t, choisi une action a_t selon sa politique $\pi(a|s)$, et reçoit une récompense immédiate r_t.
- L'objectif est d'apprendre une politique $\pi(a|s)$ qui maximise sa récompense cumulée actualisée G_t.



- À chaque pas de temps, l'agent observe l'état s_t, choisit une action a_t selon sa politique $\pi(a|s)$ et reçoit de l'environnement une récompense r_t ainsi qu'un nouvel état s_{t+1}.
- La récompense cumulée actualisée est donnée par :

$$G_t = r_t + \gamma\, r_{t+1} + \gamma^2 r_{t+2} + \cdots = \sum_{k=0}^{\infty} \gamma^k r_{t+k}$$

Avec $\gamma \in [0,1[$ est un facteur d'actualisation. Une récompense future vaut moins d'une récompense immédiate!

Apprentissage par renforcement

Le dilemme Exploration vs Exploitation:

- Une fois que l'agent a appris quelque chose, est-ce qu'il doit toujours exploiter ce qu'il sait? Ou continuer à explorer de nouvelles actions?
- C'est le dilemme exploration-exploitation fondamental en RL
- Si l'agent n'exploite que ce qu'il connaît, il risque de rester bloqué sur une solution sous-optimale. Mais s'il explore trop, il ne tire jamais profit de ce qu'il a déjà appris

La stratégie utilisée est $\epsilon - greedy$

- l'agent choisit une meilleure action connue avec une probabilité $1 - \epsilon$ (l'exploitation) et une action aléatoire avec une probabilité ϵ(l'exploration).
- En pratique, on commence avec beaucoup d'exploration au début (ϵ élevé) comme l'agent ne sait rien encore. Et on fait décroitre progressivement au fil de l'entrainement une fois l'agent commence à apprendre petit à petit

Apprentissage par renforcement

Deux grandes familles d'algorithmes:

1. Méthodes fondées sur la valeur — Q-learning, DQN

- On apprend une fonction de valeur d'action Q(s, a) : une estimation de la récompense cumulée attendue en jouant l'action a dans l'état s
- La politique est déduite de Q : on choisit l'action qui maximise Q(s, a)

2. Méthodes fondées sur la politique —PPO ou A3C

- On apprend directement une politique paramétrée $\pi(a \mid s ; \theta)$: un réseau de neurones de poids θ qui reçoit l'état et renvoie une distribution de probabilité sur les actions.

Apprentissage par renforcement

Exemple en gestion de portefeuille:

- L'objectif est d'entrainer un agent capable de décider, à chaque instant, comment allouer un capital entre plusieurs actifs financiers, A,B,C de façon à maximiser le rendement ajusté au risque sur le long terme
- L'agent apprend une politique d'allocation: étant donné l'état du marché à l'instant s_t quelle proportion du portefeuille à effectuer à chaque actif a_t

Composante	Définition
Agent	Le gestionnaire de portefeuille automatisé
Environnement	Le marché financier
Etat(s)	Snapshot du marché à l'instant t
Action (a)	Vecteur d'allocation: ex.[40%, 30%, 30%]
Récompense (r)	rendement du portefeuille, pénalité de risque, coût de transaction
Politique (π)	La stratégie apprise $\pi(s) \rightarrow a$

Apprentissage auto-supervisé

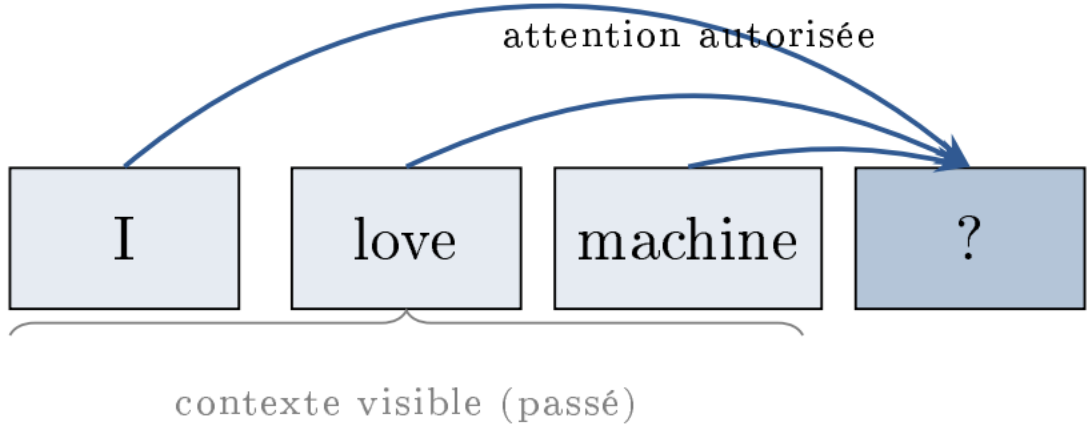
Principe

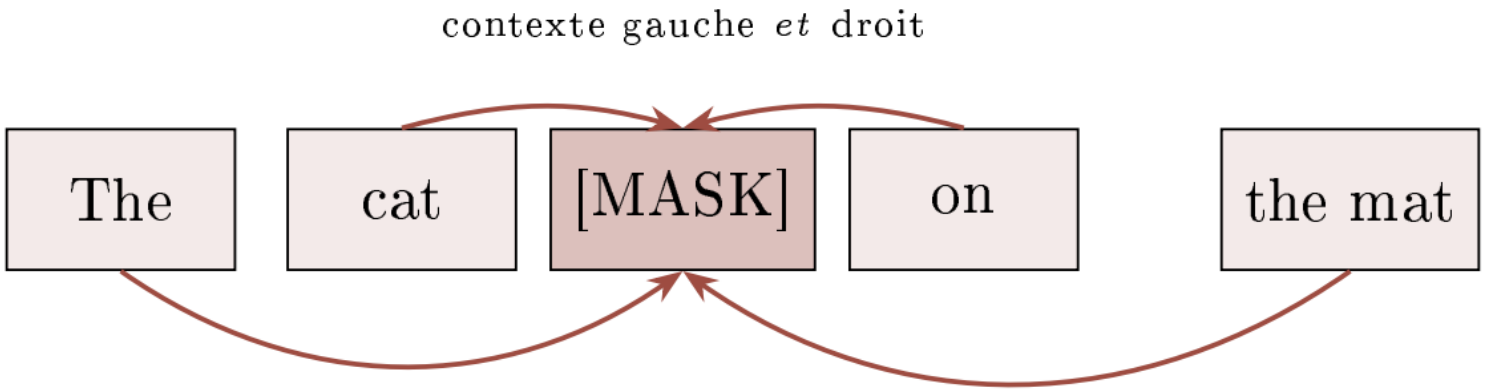
- **L'apprentissage auto-supervisé consiste à** apprendre de bonnes représentations à partir de données non étiquetées, en fabriquant automatiquement la cible à partir de la donnée elle-même (aucune annotation humaine).
- **L'idée clé est de** masquer ou transformer une partie de l'entrée, puis de demander au modèle de la prédire ou de la reconstruire → c'est la « tâche prétexte ».
- Le modèle résout des tâches prétextes dont la résolution l'oblige à développer une compréhension profonde de la structure des données.

Tâches prétextes

- **Texte (NLP) :**
  - ✓ masquer des mots et les prédire (BERT) ;
  - ✓ prédire le mot suivant (GPT).
- **Image — Reconstruction masquée :** masquer des morceaux (patchs) et les reconstruire (MAE) ; principe analogue pour l'audio (wav2vec).

Apprentissage auto-supervisé





Apprentissage auto-supervisé

Deux étapes :

1. pré-entraînement sur une tâche prétexte (sans étiquettes manuelles) : le modèle est pré-entraîné sur des corpus massifs non annotés grâce à une tâche prétexte, développant ainsi des représentations générales et riches.
2. transfert (fine-tuning) sur la tâche cible avec peu d'étiquettes : le modèle est ajusté sur un petit jeu de données annoté correspondant à la tâche réelle visée (classification, extraction d'information, détection d'objets, etc.).

C'est sur ce principe que reposent aujourd'hui les grands modèles de langage comme BERT, GPT et LLaMA

Pipeline :

1. rassembler beaucoup de données non étiquetées
2. choisir une tâche prétexte
3. pré-entraîner l'encodeur
4. geler ou affiner, puis transférer sur la tâche cible.

Deep Learning:
Apprentissage profond

L'apprentissage profonde, le Deep Learning (DNN) ou les réseaux de neurons profonds est un ensemble d'algorithms de ML imitant le fonctionnement du cerveau humain.

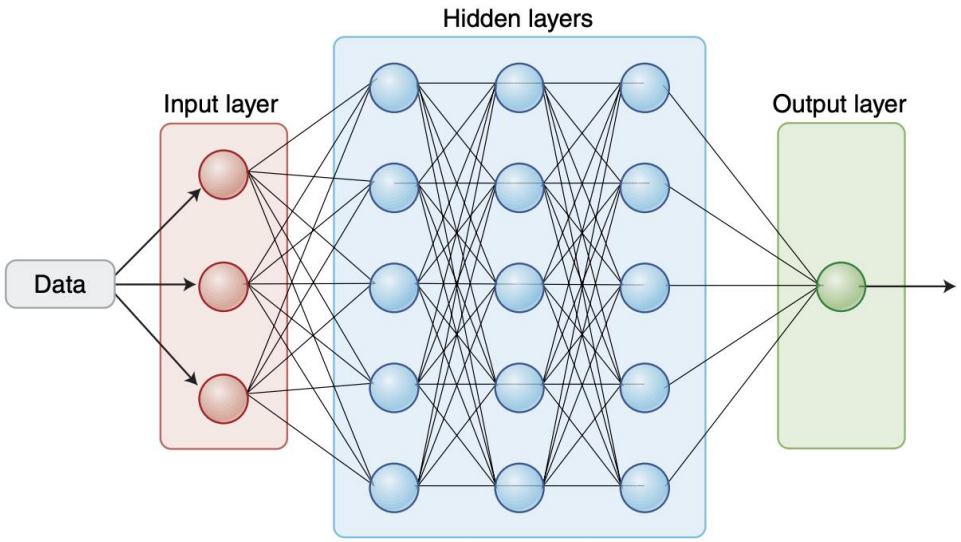


Fig. structure simplifiée d'un modèle de DNN

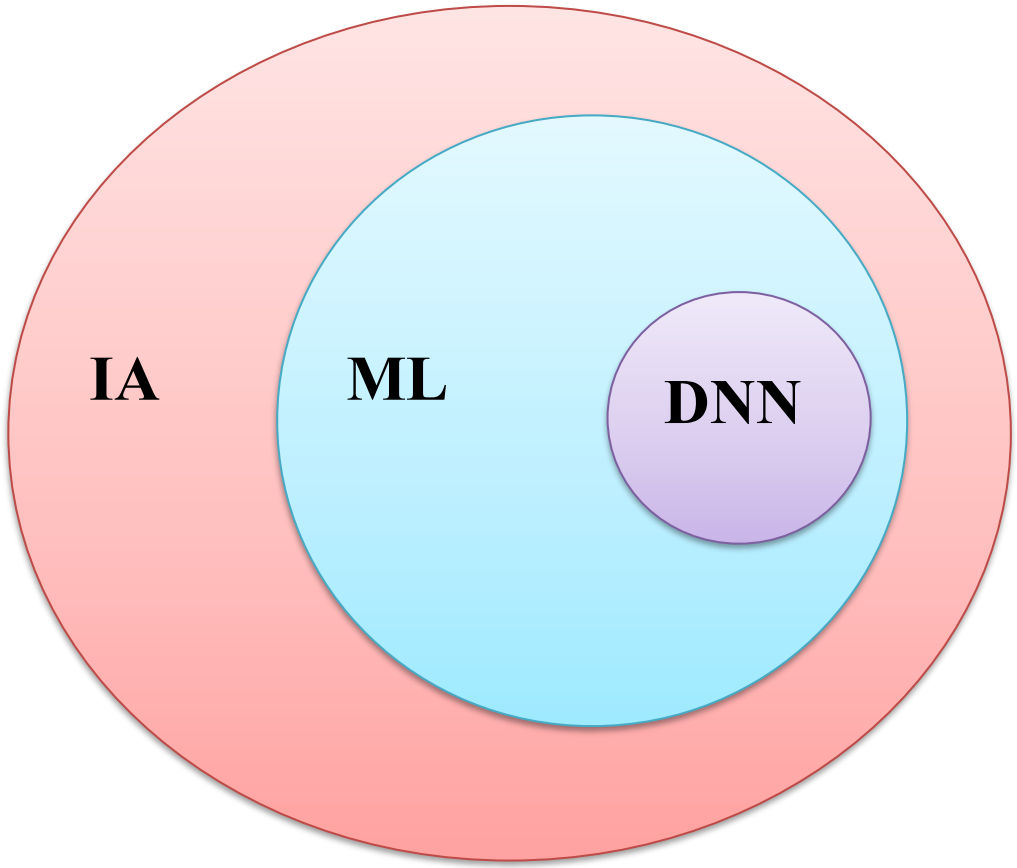


Fig. le DNN est une branche du ML

Deep Learning:
Apprentissage profond

Algorithmes classiques du Machine Learning:

Données brutes → Extraction des données →

X_1	X_2	...	X_p	Y

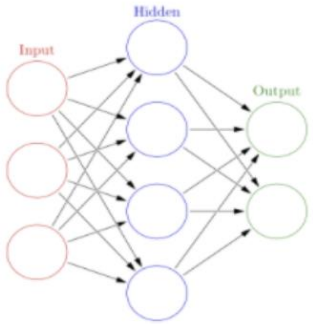
→ ML

$$Y = f(X_1, X_2, \ldots, X_p)$$

Algorithmes de Deeplearning:

DNN

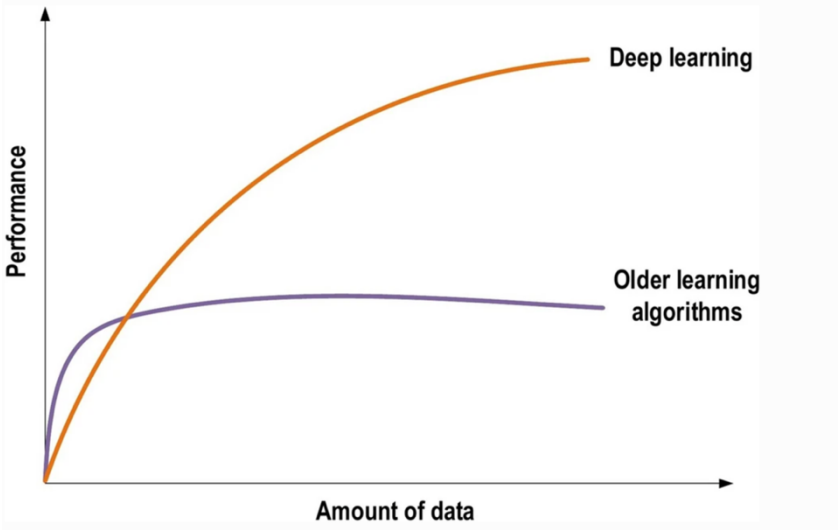
Données brutes-DB
(images, textes, signaux,…) →



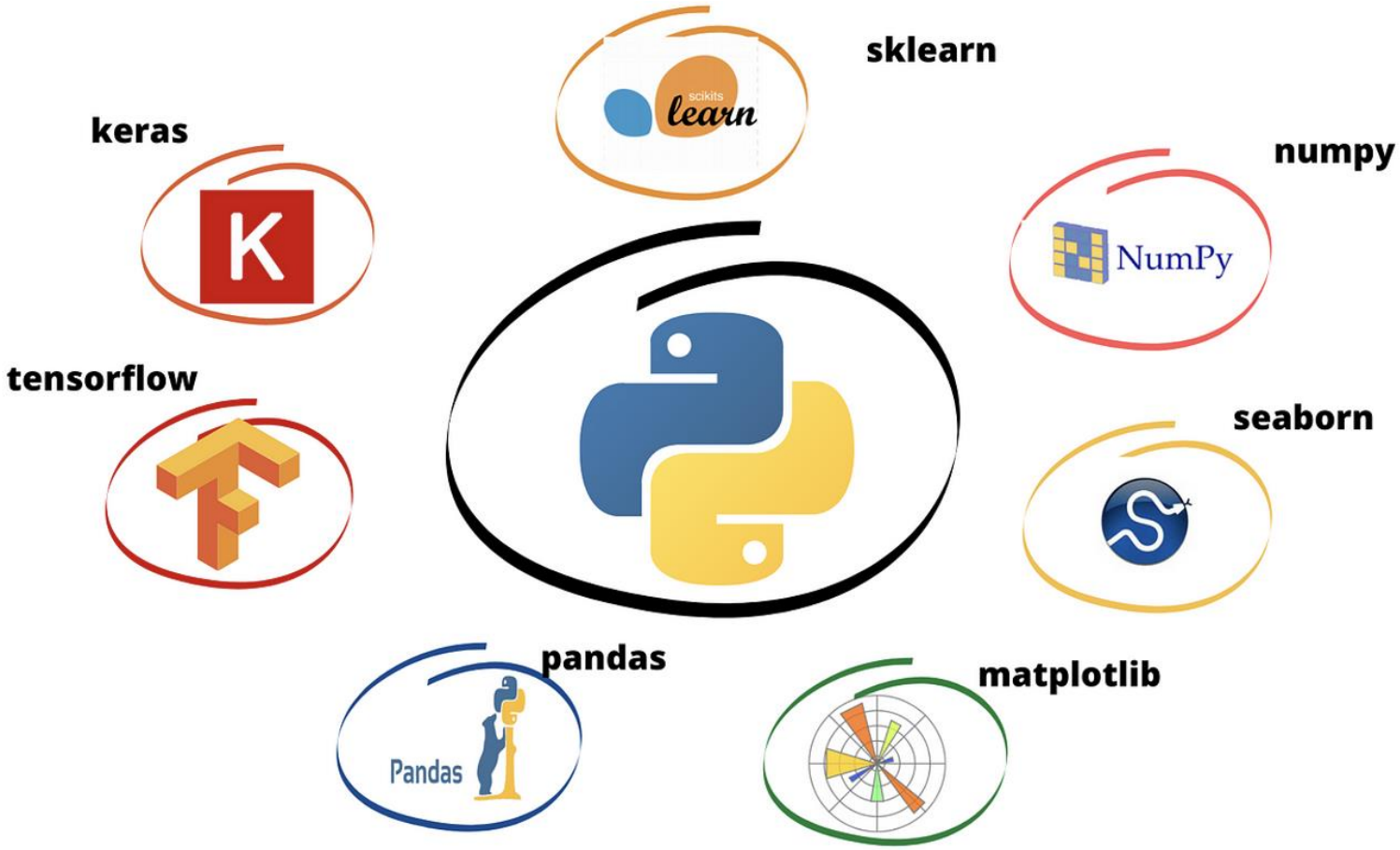
→ $\mathbf{Y = f(DB)}$

Deep Learning:
Apprentissage profond

- En bénéficiant de grandes quantités de données d'apprentissage, les réseaux de neurones profonds (DNN) ont connu un immense succès dans de nombreux domaines, notamment :
  - La classification d'images et vidéos
  - Le traitement de texte (NLP)
- Ils permettent de profiter le maximum des grandes quantités de données disponibles
- Il existe plusieurs types de réseaux neuronaux, principalement :
  1. Les réseaux de neurones artificiels (ANN)
  2. Les réseaux de neurones à convolution (CNN)
  3. Les réseaux de neurones récurrents (RNN)



Quelques Libraries Python pour la data science



- **Editeur:** Jupyter Notebook, Google Colab, VS Code

Principaux défis du Machine Learning

- Qualité et quantité des données
- Surapprentissage (*Overfitting*) et sous-apprentissage (*Underfitting*)
- Coût computationnel
- Biais et équité
- Interprétabilité et explicabilité
- Évolution des données (*Data Drift* / *Concept Drift*)
- Confidentialité et sécurité des données

Quantité et qualité des données(dataset)

- Avoir des données d'une grande qualité est un objectif prioritaire.
- Des données de mauvaise qualité peuvent avoir un impact négatif sur l'établissement qui les exploite.



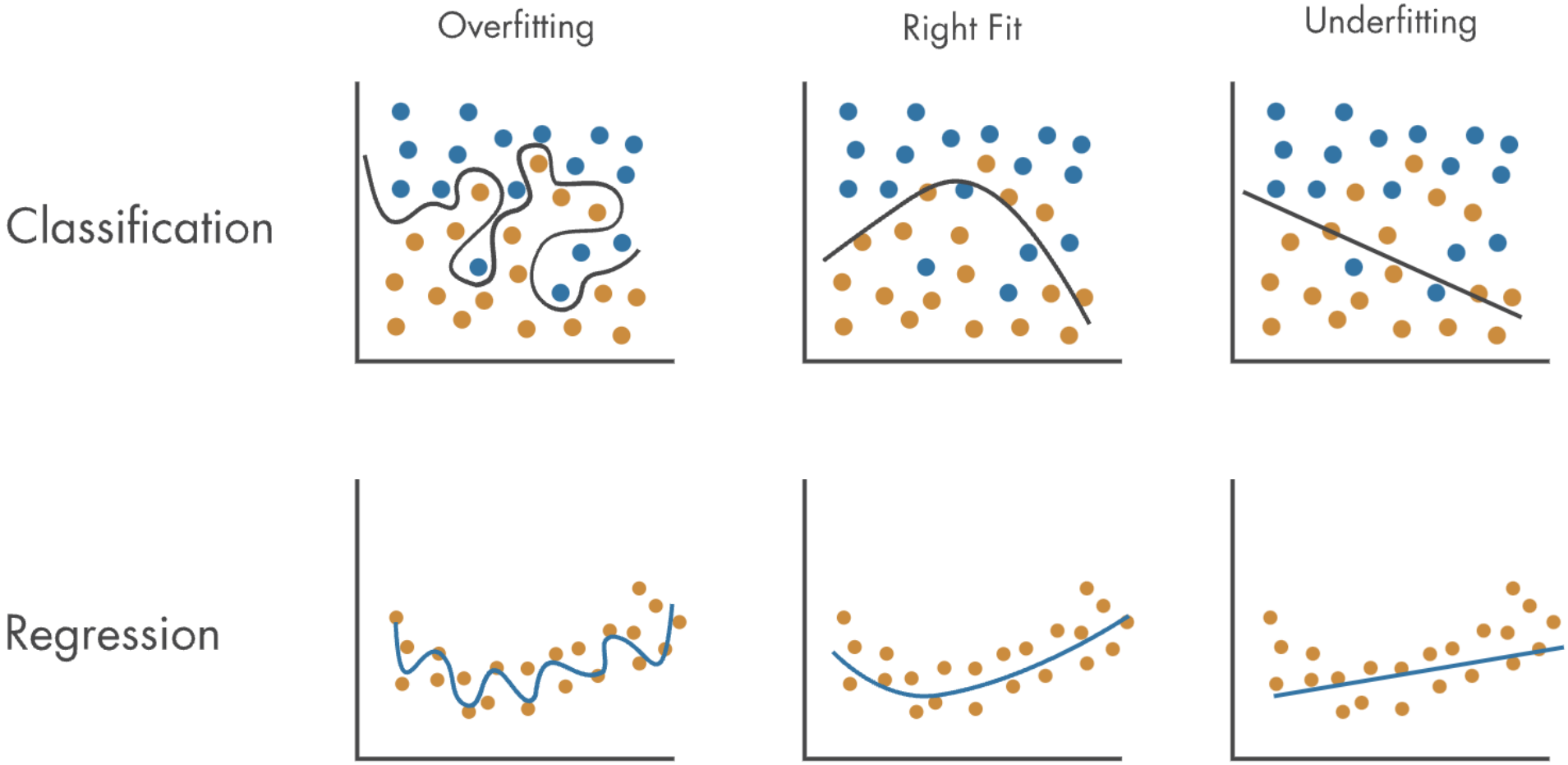
- De mauvaises segmentations
- Des estimations erronées
- Une tarification inadaptée
- Des engagements inappropriés.

Overfitting et Underfitting

- L'objectif d'un modèle de ML est de généraliser les concepts appris à des nouvelles données liées au même problème. Le modèle doit être capable de faire des prédictions sur des données dont les étiquettes non encore observées.
- Il y a deux problèmes qui peuvent influencer la capacité d'un modèle de ML à faire des bonnes prédictions sur des nouvelles données :

1. Sur-apprentissage (Overfitting the Training Data)
2. Sous-Appretissage (Underfitting the Training Data)

Overfitting et Underfitting



Overfitting et Underfitting
(Identification)

- Généralement la base de données initiale est divisée en 3 parties:

  - Training set: pour entrainer le modèle (les modèles)
  - validation set (choisir un modèle parmi les modèles entrainés)
  - Test set : tester la performance (généralisation) du modèle sélectionné

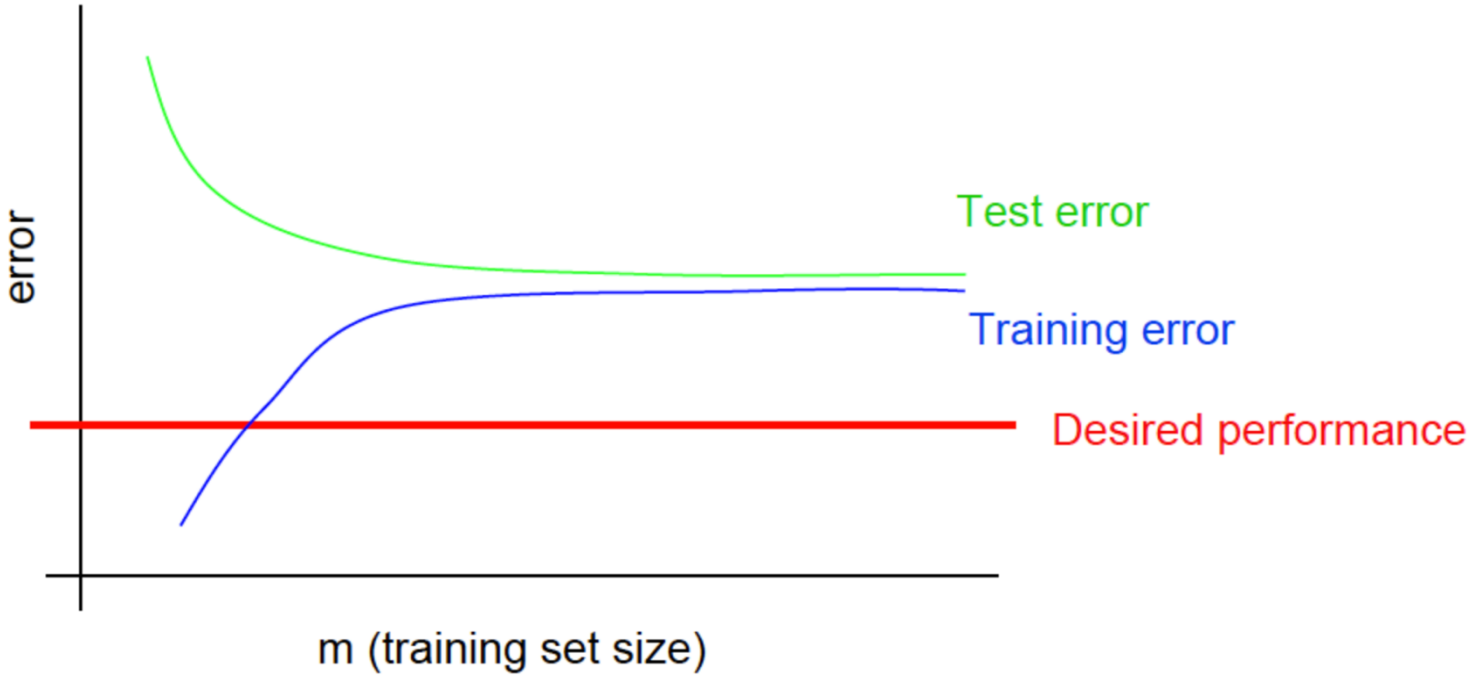
- Regarder la performance du modèle sur la base de données d'entrainement et de validation:

  - J_{train} ↑ et J_{cv} ↑ ⇒ *Underfitting (high bias)*
  - J_{train} ↓ et J_{cv} ↑ ⇒ *Overfitting(High variance)*
  - J_{train} ↓ et J_{cv} ↓ ⇒ *Modèle optimal*

Overfitting et Underfitting
(Identification)

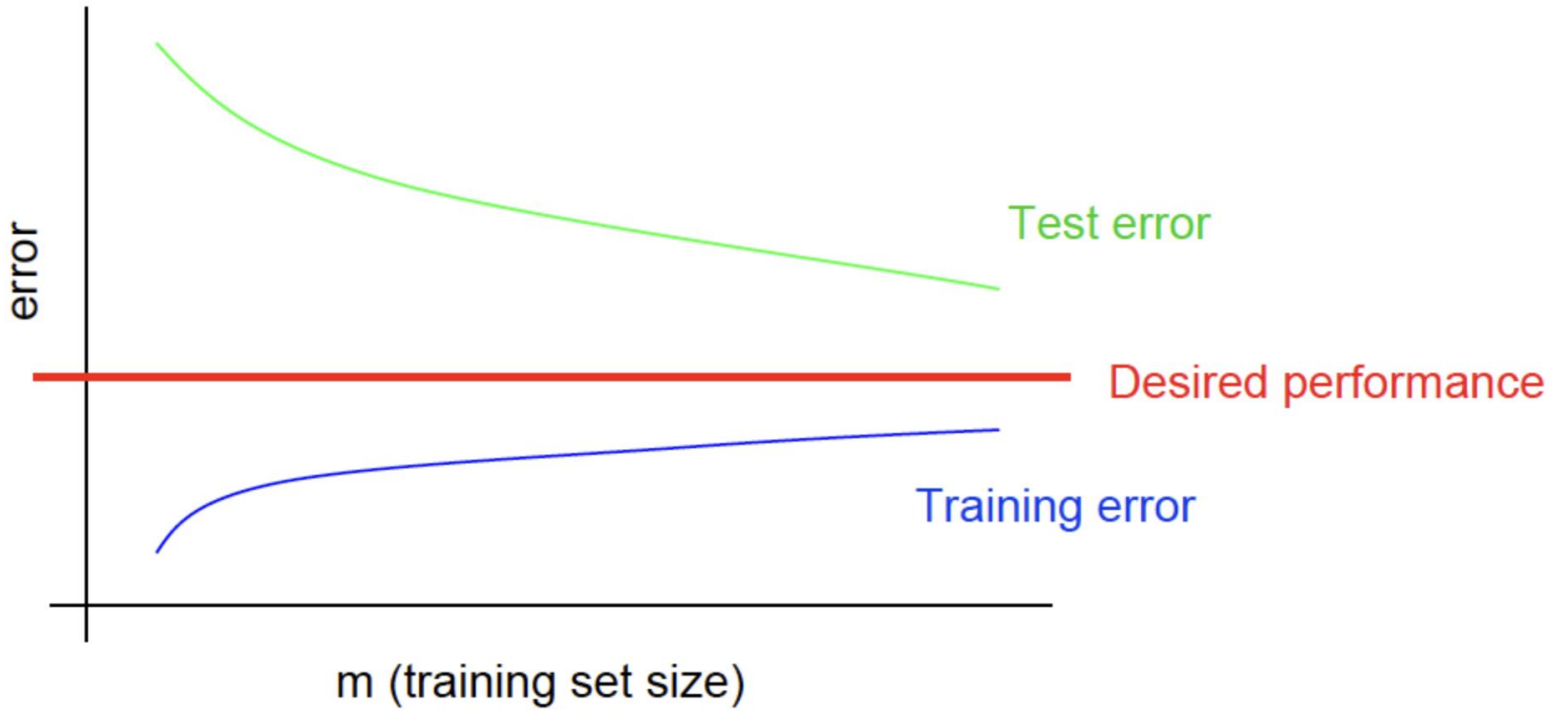
- La performance du modèle sur la base de données d'entraînement donne un aperçu du biais. Plus cette performnace est très faible(erreur elevée)plus le biais est elevé!
- Le changement d'erreur entre la base de données d'entraînement et celle de test fournit une indication de la variance. Plus cet écart est important, plus la variance est élevée.
- L'erreur est considérée elevé ou faible par rapport un niveau de base (i.e. performnace humaine)

Underfitting

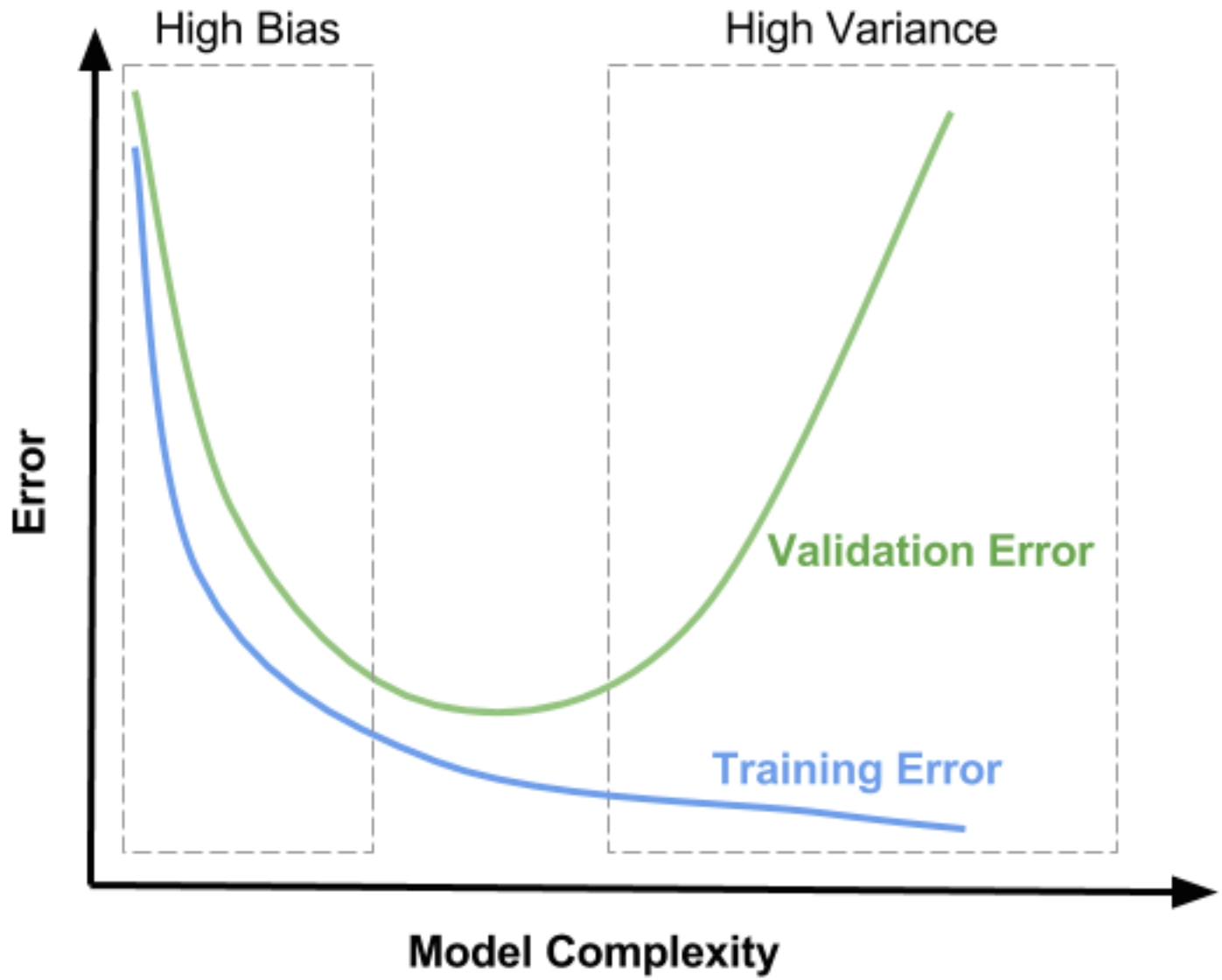


Typical learning curve for high bias (**underfitting**)

Overfitting



Typical learning curve for high variance(overfitting)


High Bias
High Variance
Error
Validation Error
Training Error
Model Complexity

Améliorer la performance d'un modèle de ML (Debugging a Machine learning algorithm)

Quand l'erreur de prédiction est élevée, il faut identifier la source de mauvaise performance, high biais ou high variance?

High bias (underfitting)

- Ajouter plus de variables explicatives
- Ajouter des variables polynomiales ($x_1^2, x_2^2, x_1 x_1 x_2, etc$)
- Jouer sur les paramètres de pénalisation (réduire la valeur de λ)
- Changer l'algorithme de ML!

High variance (overfitting)

- Augmenter la base de données d'entrainement
- Simplifier le modèle:
  - ✓ Réduire le nombre de variables explicatives (réduction de dimension)
  - ✓ Jouer sur les paramètres de pénalisation (augmenter la valeur de λ)
- Réduire le bruit des données d'entrainment (missing values)

BIAIS ALGORITHMIQUE

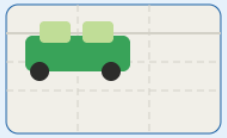
Corrélation fallacieuse : désert ≠ char

① Jeu d'entraînement

Classe 0 — Voitures


Route ensoleillée


Ville


Parking

Contexte : urbain varié

Route · Ville · Parking · ...

Classe 1 — Chars d'assaut


Désert


Désert


Désert

Contexte : toujours le désert !

Désert · Désert · Désert

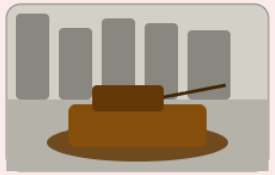
② Entraînement du modèle

Règle apprise (incorrecte) :

"Sable / paysage désertique ⇒ Char"

③ Erreurs au moment du test

Char en ville



Contexte : ville

Vrai label : char

Prédit : voiture ✗

Pas de désert détecté
→ le modèle prédit « voiture »

Voiture dans le désert



Contexte : désert

Vrai label : voiture

Prédit : char ✗

Sable détecté
→ le modèle prédit « char »

Conclusion : le modèle ne reconnaît pas vraiment les chars.

Il a appris une corrélation spurieuse (désert → char) au lieu des vraies caractéristiques visuelles.
Performances apparentes élevées à l'entraînement, mauvaise généralisation hors distribution.

Interprétabilité et explicabilité

Les modèle les plus performant (Deep Learning, méthodes ensemblistes,…) sont généralement des boites noires, ils prédisent des prédictions précises mais opaques!

L'interprétabilité et l'explicabilité peuvent être essentielles :

- Une exigence pour une conformité réglementaire (usage de l'IA)
- Pour assurer la confiance de l'utilisateur et du décideur
- Pour le débogage: détecter **les biais et erreurs**
- Une exigence pour certains domaines sensibles: médecine, justice, finance, audit

Interprétabilité et explicabilité

- Interprétabilité : intrinsèque au modèle (régression linéaire, régression logistique, arbre de décision, etc)
- Explicabilité (XAI): utiliser une méthode externe (SHAP, LIME,..):

  - Pourquoi un patient est diagnostiqué à risque?
  - Justifier le refus d'un crédit
  - Comprendre les facteurs influençant une prime d'assurance
  - Expliquer pourquoi une activité est considéré comme malveillante (suber-sécurité)

Interprétabilité et explicabilité
(Explicabilités globales vs locales)

- **Explicabilité globale:** décrit le comportement général du modèle sur l'ensemble de données

Exemple: le revenu explique 40% des décision ou l'âge explique 25% des décision)

- **Explicabilité locale:** explique une prédiction particulière

Exemple: 2 clients A et B demandent un crédit. La demande de A est refusée probablement à cause d'un revenu faible. La demande de B est refusée à cause d'un historique de défaut.

Interprétabilité et explicabilité
(principales approches)

1. Importance de variables: influence de chaque variable sur les prédictions
2. SHAP: Shaply Additive exPlanations

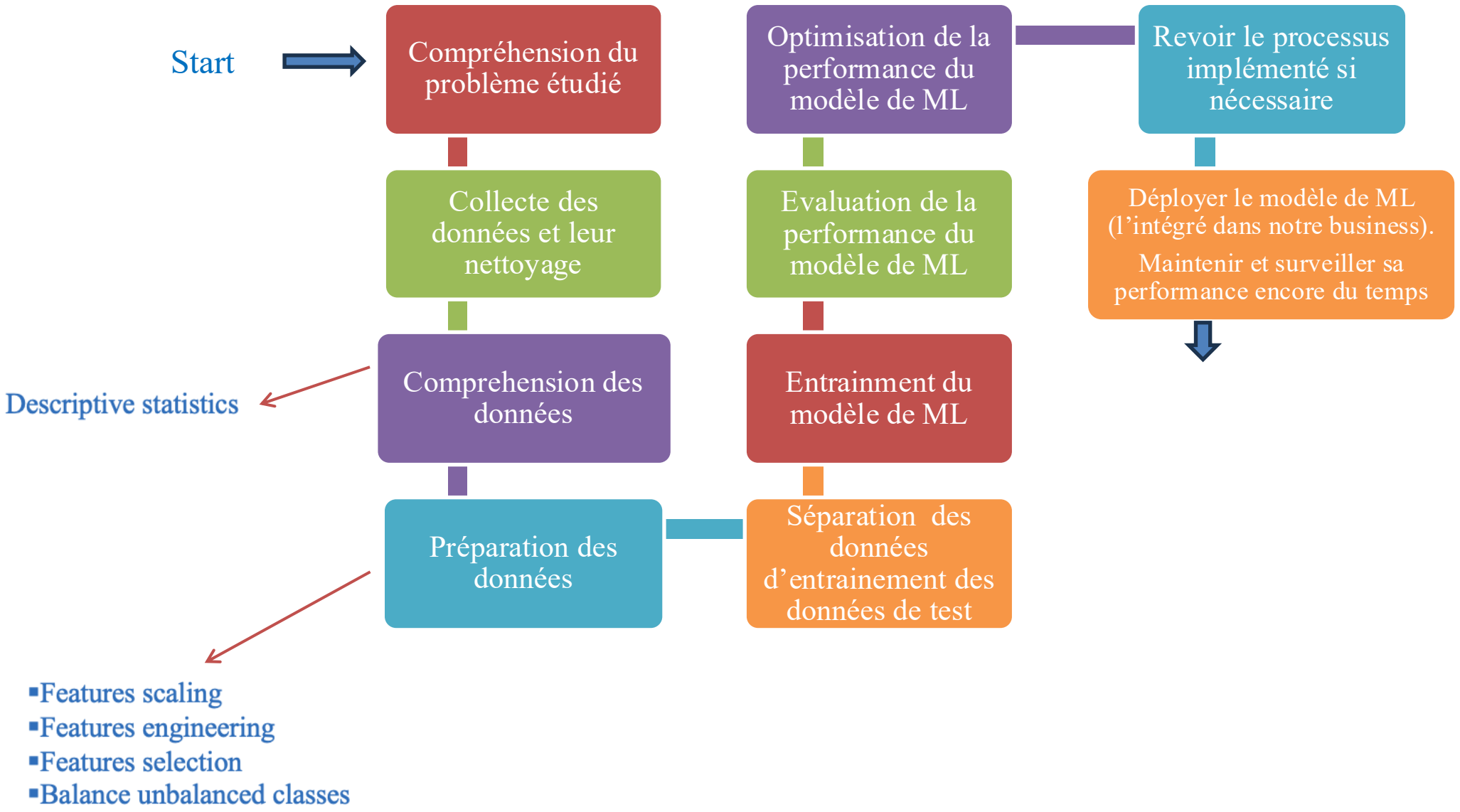
Localement: SHAP attribue à chaque variable une contribution individuelle à une prédiction en s'inspirant de la théorie des jeux coopératives.

$$prédiction_i = valeur\ de\ base + \sum_j contr_{var_j}$$

Globalement: valeur SHAP absolue moyenne pour chaque variable, plus elle est élevée plus la variable a plus d'impact.

→Elle donne également une idée sur le sens d'impact!

Le processus de développement d'un algorithme d'apprentissage supervisé


Start
Compréhension du problème étudié
Collecte des données et leur nettoyage
Comprehension des données
Descriptive statistics
Préparation des données
Features scaling
Features engineering
Features selection
Balance unbalanced classes
Séparation des données d'entrainement des données de test
Entrainment du modèle de ML
Evaluation de la performance du modèle de ML
Optimisation de la performance du modèle de ML
Revoir le processus implémenté si nécessaire
Déployer le modèle de ML (l'intégré dans notre business).
Maintenir et surveiller sa performance encore du temps

Balance Unbalanced Classes

Classes déséquilibrées (Unbalanced classes) : lorsque le nombre d'individus n'est pas le même dans chaque classe de notre échantillon, cela peut poser plusieurs problèmes.

Il est recommendé de **rééquilibrer les effectifs des différentes classes**:

- **Sur-échantillonnage (oversampling)** : on prend la classe minoritaire et on ajoute des copies d'une sélection aléatoire de ses observations jusqu'à ce qu'elle ait la même taille que la classe majoritaire.
- **Sous-échantillonnage (under-sampling)** : on prend la classe majoritaire et on supprime aléatoirement des observations jusqu'à ce qu'elle ait la même taille que la classe minoritaire.

Balance Unbalanced Classes

SMOTE ($Synthetic\ Minority\ Over-sampling\ Technique$) est une méthode de suréchantillonnage intelligent de la classe minoritaire.

- Contrairement au simple duplicata d'exemples minoritaires, SMOTE crée de nouveaux échantillons synthétiques en interpolant les caractéristiques de plusieurs voisins proches
- Étapes de SMOTE :

  Pour chaque échantillon de la classe minoritaire, on choisit aléatoirement k voisins les plus proches (généralement $k = 5$).

  On en sélectionne un au hasard parmi ces voisins.

  On crée un nouvel échantillon synthétique selon la formule :

$$\boldsymbol{x_{new}} = \boldsymbol{x_i} + \boldsymbol{\lambda}(\boldsymbol{x_{voisin}} - \boldsymbol{x_i})$$

  où λ est un nombre aléatoire entre 0 et 1. Cela génère un point situé entre deux exemples minoritaires réels, enrichissant l'espace de décision du modèle

Balance Unbalanced Classes

SMOTE est appliqué uniquement sur le jeu d'entraînement, jamais sur tout le $dataset$, pour éviter la data $leakage$.

- $Accuracy$ n'est souvent pas pertinente sur les données déséquilibrées.
- Privilégie des métriques adaptées : $Precision, Recall, F1-score, ROC\ ou\ AUC$

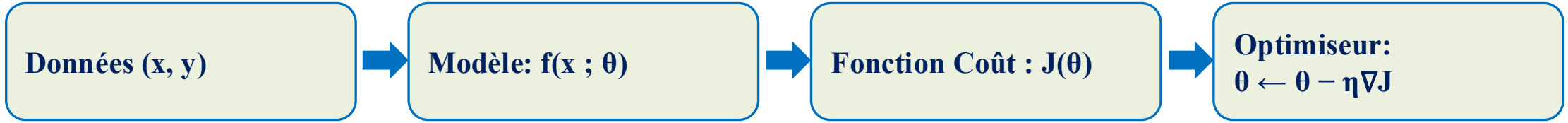
Remarque :

ADASYN ($Adaptive\ Synthetic\ Sampling$) est une méthode de suréchantillonnage adaptatif conceptuellement proche de SMOTE, mais elle introduit une pondération adaptative qui se concentre davantage sur les exemples minoritaires difficiles à apprendre.

Apprentissage supervisé
(Régression et classification supervisée)

Comment un modèle apprend

(Données → modèle → coût → optimisation)



↺ *on recommence : nouvelle passe sur les données (epoch)*

- **Données d'entraînement** : un ensemble d'exemples (x, y) supposés représentatifs de la tâche : le modèle ne connaîtra le problème qu'à travers eux.
- **Modèle** : une fonction f(x ; θ) paramétrée par θ, les poids. Apprendre, c'est choisir θ optimal
- **Fonction de coût** J(θ) : elle chiffre l'écart entre prédictions et vérité (erreur quadratique en régression, entropie croisée en classification). C'est la quantité à minimiser.
- **Optimisation** : utiliser un algorithme d'optimisation pour trouver θ qui minimise J(θ):
- On répète jusqu'à convergence : $\theta \leftarrow \theta - \eta \cdot \nabla J(\theta)$
- **Algorithmes — GD, SGD / mini-batch, Momentum, Adam** (standard en deep learning) .

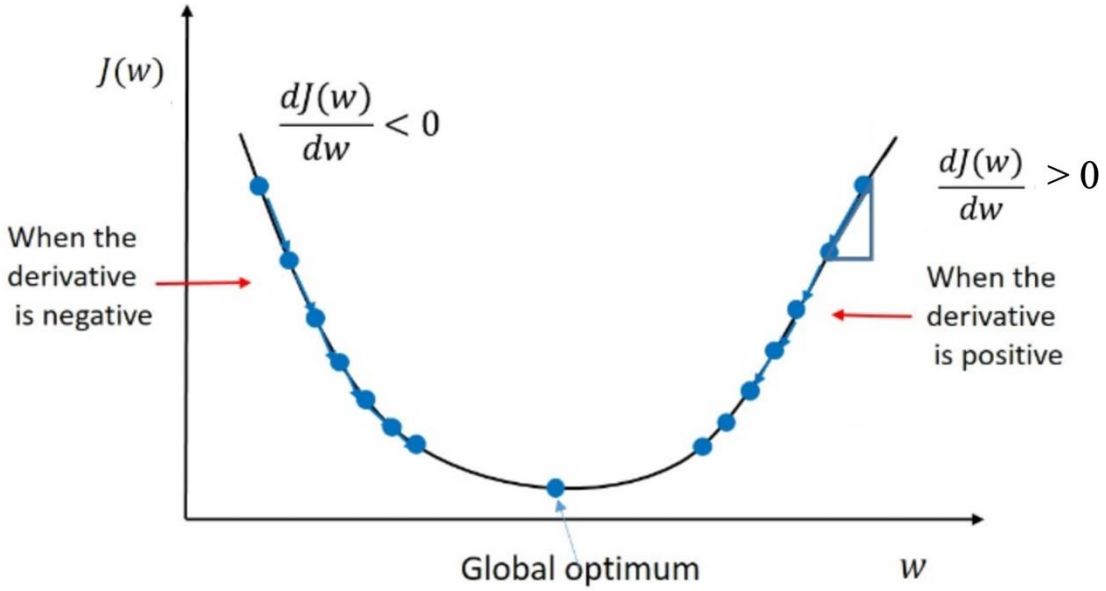
Algorithmes d'optimisation utilisés:

Descente de Gradient Batch (GD)

- Répéter jusqu'à convergence (pas de changement de w):

$$w_t = w_{t-1} - \alpha \frac{\partial J(w_{t-1})}{\partial w}$$

Avec $\alpha > 0$ est le taux d'apprentissage (Learning rate)



Algorithmes d'optimisation utilisés:

Descente de Gradient Batch (GD)

- Pour optimiser une fonction par rapport à des paramètres $w = (w_1, w_1, \dots, w_p)$, l'algorithme de descente de gradient est donné par:
- Répéter simultanément jusqu'à convergence (avoir l'optimum):

$$w_1 = w_1 - \alpha \frac{\partial J(w)}{\partial w_1}$$

$$w_2 = w_2 - \alpha \frac{\partial J(w)}{\partial w_2}$$

$$\dots\dots$$

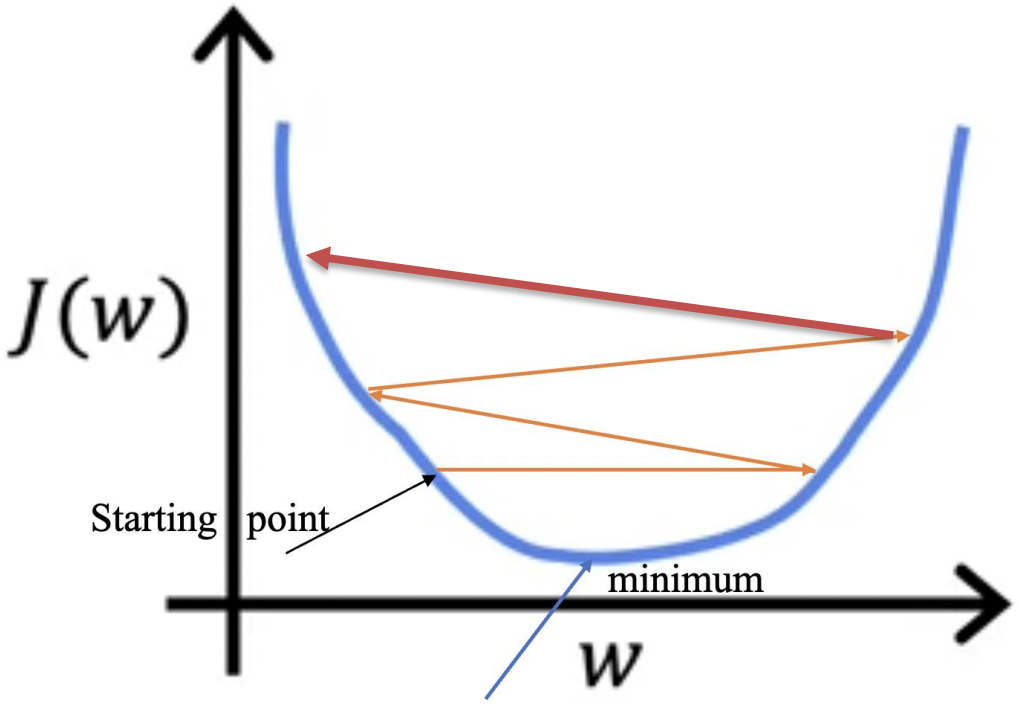
$$w_p = w_p - \alpha \frac{\partial J(w)}{\partial w_p}$$

$$\Leftrightarrow \quad W = W - \alpha \nabla J$$

- J est la fonction coût
- ∇J dépend de la data d'entrainement X entière

Algorithmes d'optimisation utilisés:

Descente de Gradient Batch (GD)



exemple avec un α très grande

Algorithmes d'optimisation utilisés:
Variantes de la Descente de Gradient

- La formulation classique de Déscente de gradient classique (DG Bach) est lente sur grands datasets. Chaque mise à jour exige l'utilisation de toute la BDD d'entrainement (X)
- D'autre générations d'amélioration ont about à Adam, l'optimiseur par défaut du deep larning moderne:
  1. Descente de gradient stochastique (SGD): remplace le gradient exact par une estimation bruité calculé sur un mini-bach (ou un seul exemple), rendant chaque mise à jour beaucoup plus rapide.
  2. Adam: permet un un taux d'apprentissage spécifique pour chaque paramètre.

Algorithmes d'optimisation utilisés:
ADAM

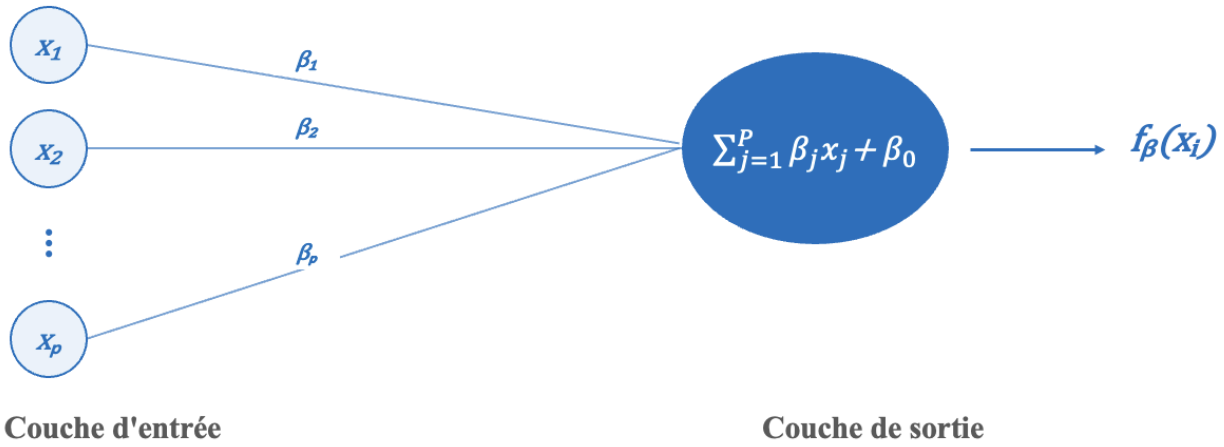
- **Ada**ptive **M**oment Estimation(ADAM):
  - Selon le comportement de la descente de gradient (DG), ADAM ajuste automatiquement le taux d'apprentissage, ce qui lui confère une grande adaptabilité aux différents types de données et aux dynamiques des problèmes d'optimisation.
  - Si la DG conserve la même direction au fil du temps, le taux d'apprentissage est augmenté, tandis que s'il change constamment de direction, le taux d'apprentissage est réduit pour assurer une convergence efficace.
  - ADAM offre généralement une meilleure performance en termes de convergence rapide et d'adaptabilité aux données, mais cela peut nécessiter plus de ressources computationnelles en raison de la nécessité de stocker des historiques de gradients.

Régression

Data:

x_1	x_2	...	x_p	y
x_{11}	x_{12}	...	x_{1p}	y_1
x_{21}	x_{22}	...	x_{2p}	y_2
⋮	⋮	⋮	⋮	⋮
x_n	x_{n2}	...	x_{np}	y_n

Modèle: $f_\beta(x_i) = \sum_{j=1}^{P} \beta_j x_{ij} + \beta_0$



Régression

Fonction coût ou Loss(MSE):

$$J(\beta) = \frac{1}{2n}\sum_{i=1}^{n}\left(y_i - f(x_i)\right)^2 = \frac{1}{2n}\|y - X\beta\|_2^2$$

Avec:

$$y = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix}, X = \begin{pmatrix} 1 & x_{11} & x_{12} & \cdots & x_{1p} \\ 1 & x_{21} & x_{22} & \cdots & x_{2p} \\ \vdots & \vdots & \vdots & \cdots & \vdots \\ 1 & x_{n1} & x_{n2} & \cdots & x_{np} \end{pmatrix}, \beta = \begin{pmatrix} \beta_0 \\ \beta_1 \\ \vdots \\ \beta_p \end{pmatrix}$$

- Estimation des paramètres : $\hat{\beta} = \text{argmin}_\beta J(\beta)$ → via Descent de gradient ou ses variants

Régression
(Estimation des paramètres)

Répéter simultanément jusqu'à convergence :

$$\{ \mathrm{B} = \mathrm{B} - \alpha \nabla J \}$$

Avec:

$$\nabla = \frac{1}{n} X^T (X\mathrm{B} - y)$$

- α est le taux d'apprentissage!

Régression
(Estimation des paramètres)

Pseudocode :

1. Initialiser $\mathrm{B}^{(0)}$ ($z\acute{e}ro\ ou\ al\acute{e}atoire$), $fixer\ \alpha$, tolérance ϵ et le nombre d'itérations maximal T
2. Pour tout $t = 0,1, \ldots, T-1$:
   - $g = \frac{1}{n} X^T (X\beta - y)$
   - $\mathrm{B} = \mathrm{B} - \alpha g$
   - **Arrêt** si $\left|\left|g\right|\right|_2 \leq \epsilon$

Régression avec Régularisation

▪La régression linéaire de sa forme classique présente certaines limites quand les variables explicatives sont fortement corrélées ou leur nombre dépasse celui des observations.

▪ La solution n'est plus unique et la variance des coefficients explose.

▪Pour contrôler la complexité du modèle de régression (problème de surapprentissage) on introduit un terme de régularisation qui pénalise l'amplitude des coefficients et stabilise l'estimation des paramètres:

- Ridge: $Min_{\beta}.J(B) + \lambda||\beta||_2^2$
- Lasso : $Min_{\beta}.J(B) + \lambda||\beta||_1$
- Elastic Net: $Min_{\beta}.J(B) + \lambda\,[\,\alpha||\beta||_1 + (1-\alpha)||\beta||_2^2]$

Régression avec Régularisation

Remarque:

- Plus λ est grand plus on donne de l'importance à $\lambda\left|\left|\beta\right|\right|_2^2$ par rapport à $\left|\left|Y - X\beta\right|\right|_2^2$, donc minimiser le problème revient à minimiser $\left|\left|\beta\right|\right|_2^2$ c'est-à-dire les β_j

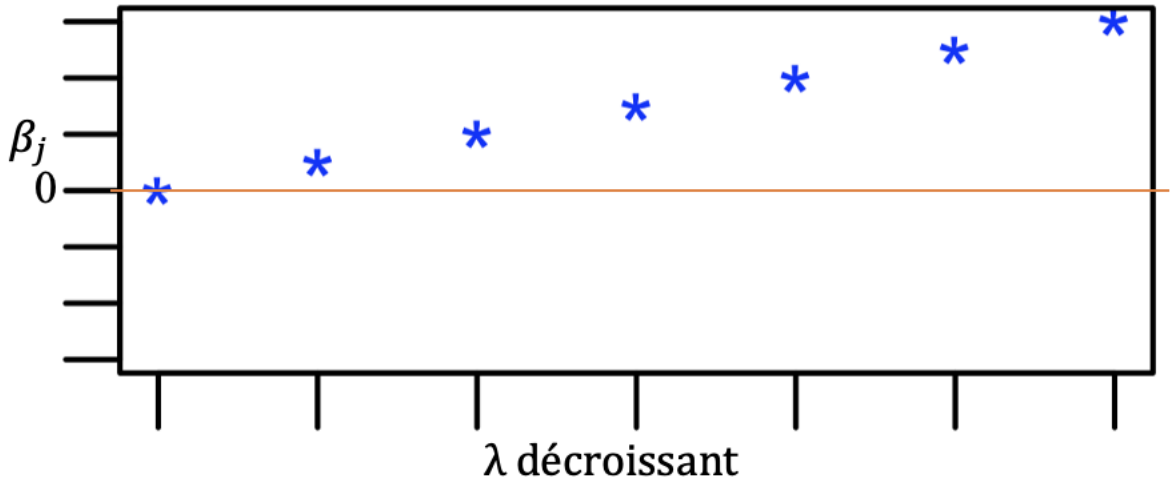


Fig. Effet de la régularisation sur la variable X_j

N.B: même raisonnement pour $||\beta||$

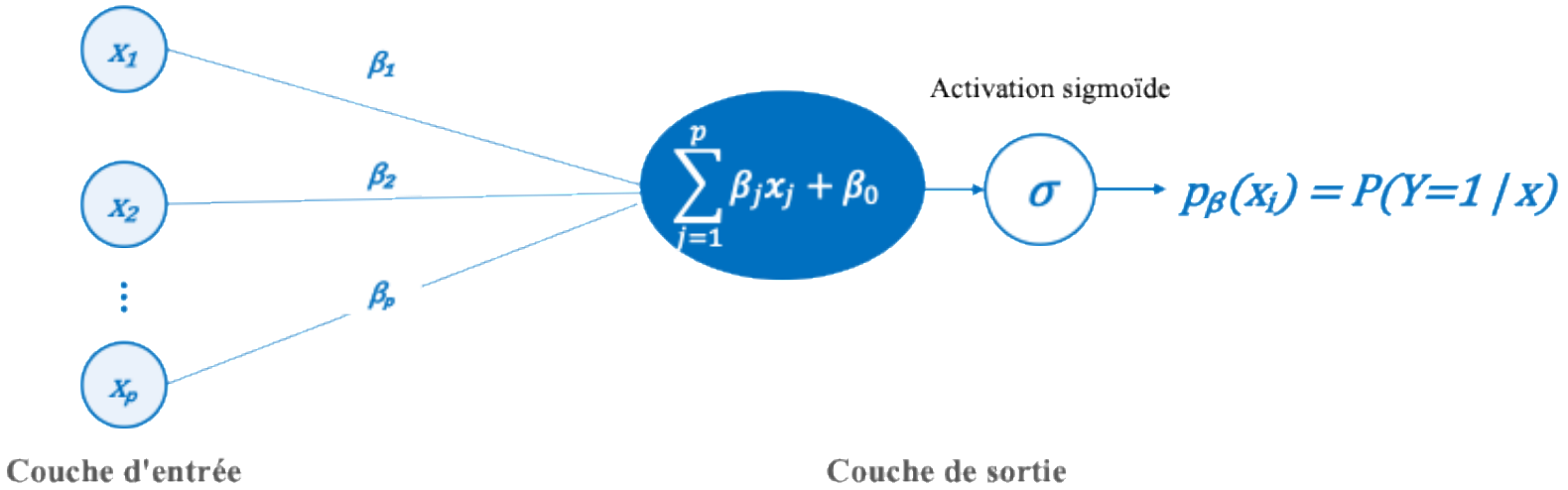
Classification binaire

- **Data:**

x_1	x_2	…	x_p	y
x_{11}	x_{12}	…	x_{1p}	y_1
x_{21}	x_{22}	…	x_{2p}	y_2
⋮	⋮	⋮	⋮	⋮
x_n	x_{n2}	…	x_{np}	y_n

$y_i \in \{0, 1\}$

- **Modèle:** $\mathrm{p}(y_i = 1|X = x_i) = \sigma\left(\sum_{j=1}^{p} \beta_j x_j + \beta_0\right) = \frac{1}{1+e^{-\left(\sum_{j=1}^{p} \beta_j x_j + \beta_0\right)}} = p_\beta(x_i)$



Classification binaire

- **Fonction coût (BCE) :**

$$J(B) = -\frac{1}{n}\sum_{i=1}^{n} \mathrm{y_i} \ln\left(p_\beta(x_i)\right) + (1 - y_i) \ln\left(1 - p_\beta(x_i)\right)$$

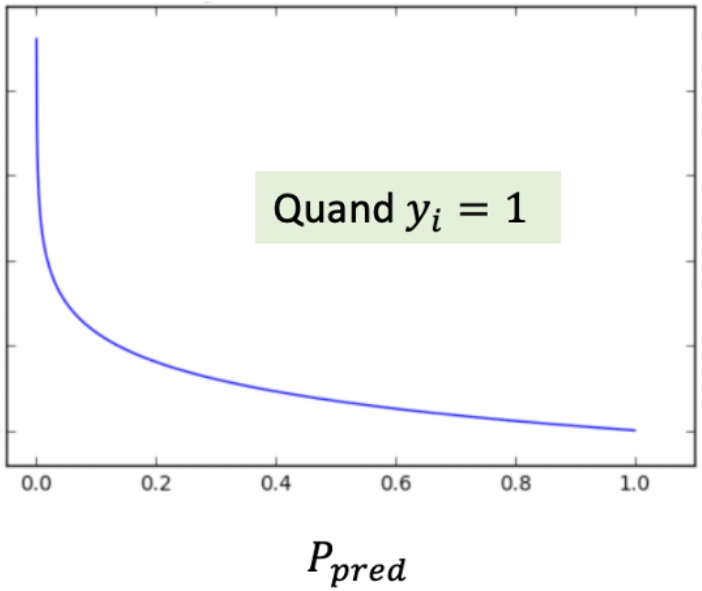
- **Estimation des paramètres :**

$$\hat{\beta} = \mathrm{argmin}_\beta J(\beta)$$

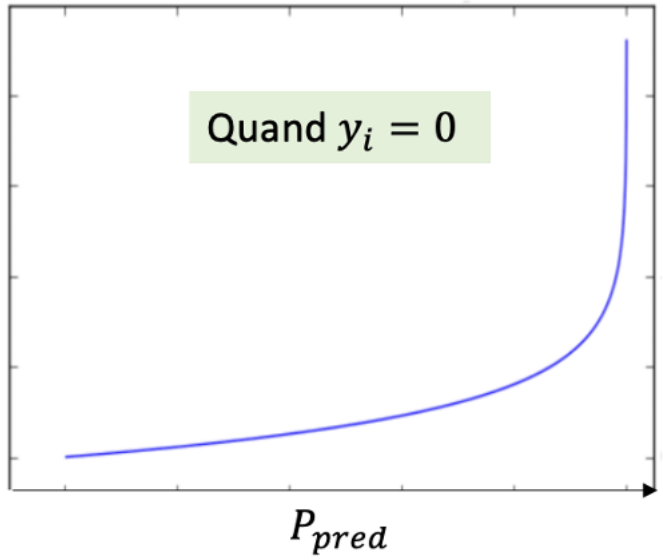
→ via Descent de gradient ou ses variants

Classification binaire

$-\ln(P_{pred})$



$-\ln(1 - P_{pred})$



Classification binaire
Estimation des paramètres

Répéter jusqu'à convergence:

$$\{ \mathrm{B} = \mathrm{B} - \alpha \nabla_{\beta} J(\mathrm{B}) \}$$

Avec :

$$\nabla_{\beta} J(\mathrm{B}) = \frac{1}{n} X^{T} (\hat{y} - y)$$

$$= \frac{1}{n} X^{T} (\sigma(X\mathrm{B}) - y)$$

$$\sigma(XB) = \frac{1}{1 + \exp(-X\mathrm{B})} = \begin{pmatrix} p(x_1) \\ p(x_2) \\ \vdots \\ p(x_n) \end{pmatrix}$$

$$\text{et } p(x_i) = p(Y_i = 1 | X = x_i)$$

Classification multi-classes (Régression softmax)

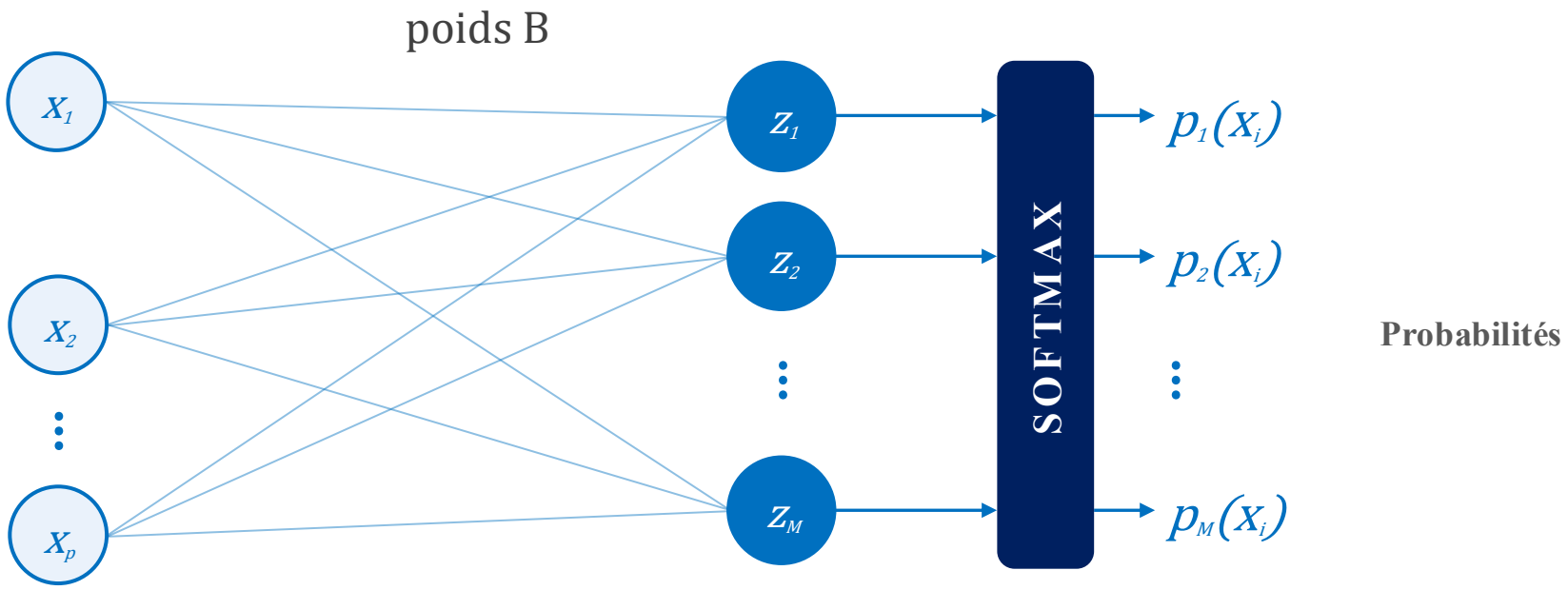
- **Data**

x_1	x_2	...	x_p	y_{i1}	y_{i2}	...	y_{iM}
x_{11}	x_{12}	...	x_{1p}	y_{11}	y_{12}		y_{1M}
x_{21}	x_{22}	...	x_{2p}	⋮	⋮	⋮	⋮
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
x_n	x_{n2}	...	x_{np}	y_{n1}	y_{n2}		y_{nM}

M est le nombre de classe et $\sum_k^M y_{ik} = 1$

Classification multi-classes (Régression softmax)

- **Modèle:** $p(Y_i = k|X = x_i) = \frac{\exp(z_k)}{\sum_{m=1}^{M} \exp(z_m)} = p_k(x_i)$



$$B = \begin{pmatrix} \beta_{10} & \beta_{20} & \beta_{30} & \cdots & \beta_{M0} \\ \beta_{11} & \beta_{21} & \beta_{31} & \cdots & \beta_{M1} \\ \vdots & \vdots & \vdots & \cdots & \vdots \\ \beta_{1p} & \beta_{2p} & \beta_{3p} & \cdots & \beta_{Mp} \end{pmatrix}$$

$z_k = \beta_{k,0} + \sum_{j=1}^{p} \beta_{k,j} x_{ij} \quad \forall k \in \{1, \dots, M\}$

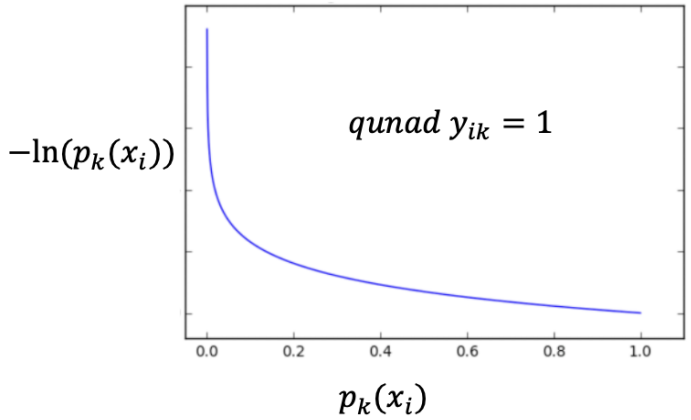
N.B: chaque classe a ses propres paramètres!

➢ *Régle de décision:* $\hat{y}_i = \boldsymbol{argmax_k p_{k(x_i)}}$

Classification multi-classes (Régression softmax)

Fonction coût (CCE) :

$$J(\mathrm{B}) = -\frac{1}{n}\sum_{i=1}^{n}\sum_{k=1}^{M} y_{ik}\ln\left(p_k(x_i)\right)$$



$$y_{ik} = \begin{cases} 1 & \text{si } k \text{ est la vraie classe à prédire} \\ 0 & \text{sinon} \end{cases}$$

Cette forme de $J(\mathrm{B})$ est appelée « Categorical Cross Entropy »

	y_{i1}	…	y_{ik}	…	y_{iM}
Vraie y_i	0	…	1	…	0
Prédiction	$p_1(x_i)$		$p_k(x_i)$		$p_M(x_i)$

Classification multi-classes (Régression softmax)

Une autre forme de $J(\mathrm{B})$ est « Sparse Ctegorical Cross Entropy »

$$J(\mathrm{B}) = -\frac{1}{n}\sum_{i=1}^{n} \ln\left(p_k(x_i)\right)$$

- Elle utilisé quand la variable y_i est un entier représentant l'indice de la vraie classe au lieu d'un vecteur '$one - hot - encoded$ ' comme précédemment
- $\forall\ y_i \in \{1,2,\dots,M\}$ on prédit $(p_1(x_i),\dots,p_M(x_i))$
- $p_k(x_i)$ est la probabilité correspondante à la vraie valeur y_i

Régression Softmax
(Estimation des paramètres)

Répéter jusqu'à convergence:

$\{ \mathrm{B} = \mathrm{B} - \alpha \nabla_{\beta} J(\mathrm{B}) \}$

Avec :

$$\nabla_{\beta} J(\mathrm{B}) = \frac{1}{n} X^T (\hat{y} - y) = \frac{1}{n} X^T (Softmax(XB) - y))$$

$$X = \begin{pmatrix} 1 & x_{11} & x_{12} & \cdots & x_{1p} \\ 1 & x_{21} & x_{22} & \cdots & x_{2p} \\ \vdots & \vdots & \vdots & \cdots & \vdots \\ 1 & x_{n1} & x_{n2} & \cdots & x_{np} \end{pmatrix} \quad y = \begin{pmatrix} y_{11} & y_{12} & y_{13} & \cdots & y_{1M} \\ y_{21} & y_{22} & y_{23} & \cdots & x_{2M} \\ \vdots & \vdots & \vdots & \cdots & \vdots \\ y_{n1} & y_{n2} & y_{n\,3} & \cdots & y_{nM} \end{pmatrix}$$

Avec: $y_{ij} =$ **1** si j est la vraie classe et **0** sinon.

Régression Softmax
(Estimation des paramètres)

$$X = \begin{pmatrix} 1 & x_{11} & x_{12} & \cdots & x_{1p} \\ 1 & x_{21} & x_{22} & \cdots & x_{2p} \\ \vdots & \vdots & \vdots & \cdots & \vdots \\ 1 & x_{n1} & x_{n2} & \cdots & x_{np} \end{pmatrix}, \qquad \mathrm{B} = \begin{pmatrix} \beta_{10} & \beta_{20} & \beta_{30} & \cdots & \beta_{M0} \\ \beta_{11} & \beta_{21} & \beta_{31} & \cdots & \beta_{M1} \\ \vdots & \vdots & \vdots & \cdots & \vdots \\ \beta_{1p} & \beta_{2p} & \beta_{3p} & \cdots & \beta_{Mp} \end{pmatrix},$$

$$\mathrm{XB} = \begin{pmatrix} x_1^T\beta_1 & x_1^T\beta_2 & x_1^T\beta_3 & \cdots & x_1^T\beta_M \\ x_2^T\beta_1 & x_2^T\beta_2 & x_2^T\beta_3 & \cdots & x_2^T\beta_M \\ \vdots & \vdots & \vdots & \cdots & \vdots \\ x_n^T\beta_1 & x_n^T\beta_2 & x_n^T\beta_3 & \cdots & x_n^T\beta_M \end{pmatrix}, \qquad \mathrm{Softmax(XB)} = \begin{pmatrix} p_1(x_1) & p_2(x_1) & p_3(x_1) & \cdots & p_M(x_1) \\ p_1(x_2) & p_2(x_2) & p_3(x_2) & \cdots & p_M(x_2) \\ \vdots & \vdots & \vdots & \cdots & \vdots \\ p_1(x_n) & p_2(x_n) & p_3(x_n) & \cdots & p_M(x_n) \end{pmatrix}$$

Avec $p_j(x_i) = softmax\left(x_i^T\beta_j\right) = \dfrac{e^{x_i^T\beta_j}}{\sum_{k=1}^{M} e^{x_i^T\beta_k}} = p(Y_i = j|X = x_i), k \in \{1,2,\dots,M\}$

C'est la probabilité que l'individu i appartient à la classe j

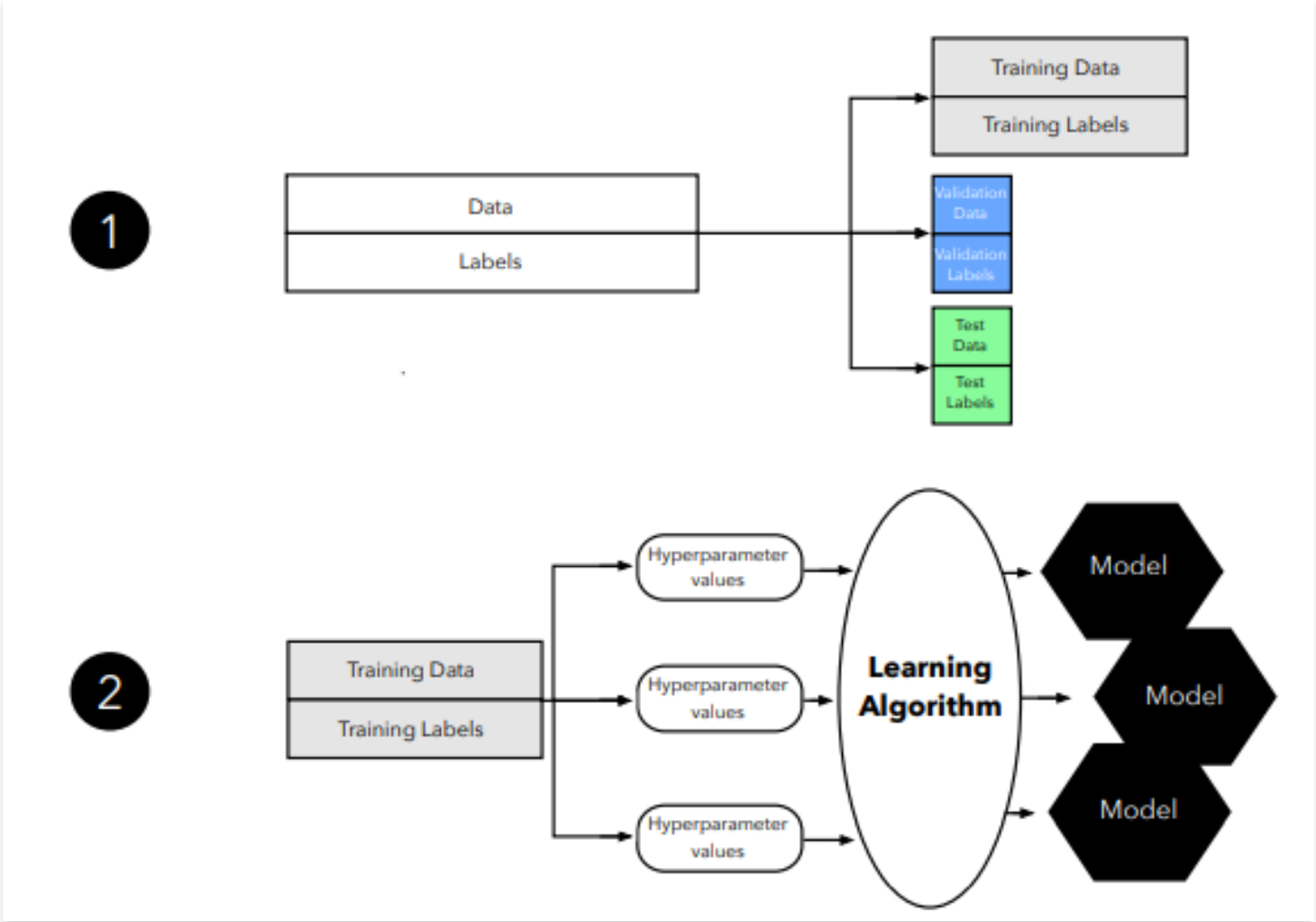
Evaluation de la performance d’un modèle de classification et estimation de ses hyperparamètres

Three-Way Holdout method

La base de données initiales est devisée en 3 parties, training, validation et test :

1. Entrainer plusieurs modèles avec différentes valeurs des hyperparamètres à l'aide de la base de données d'apprentissage

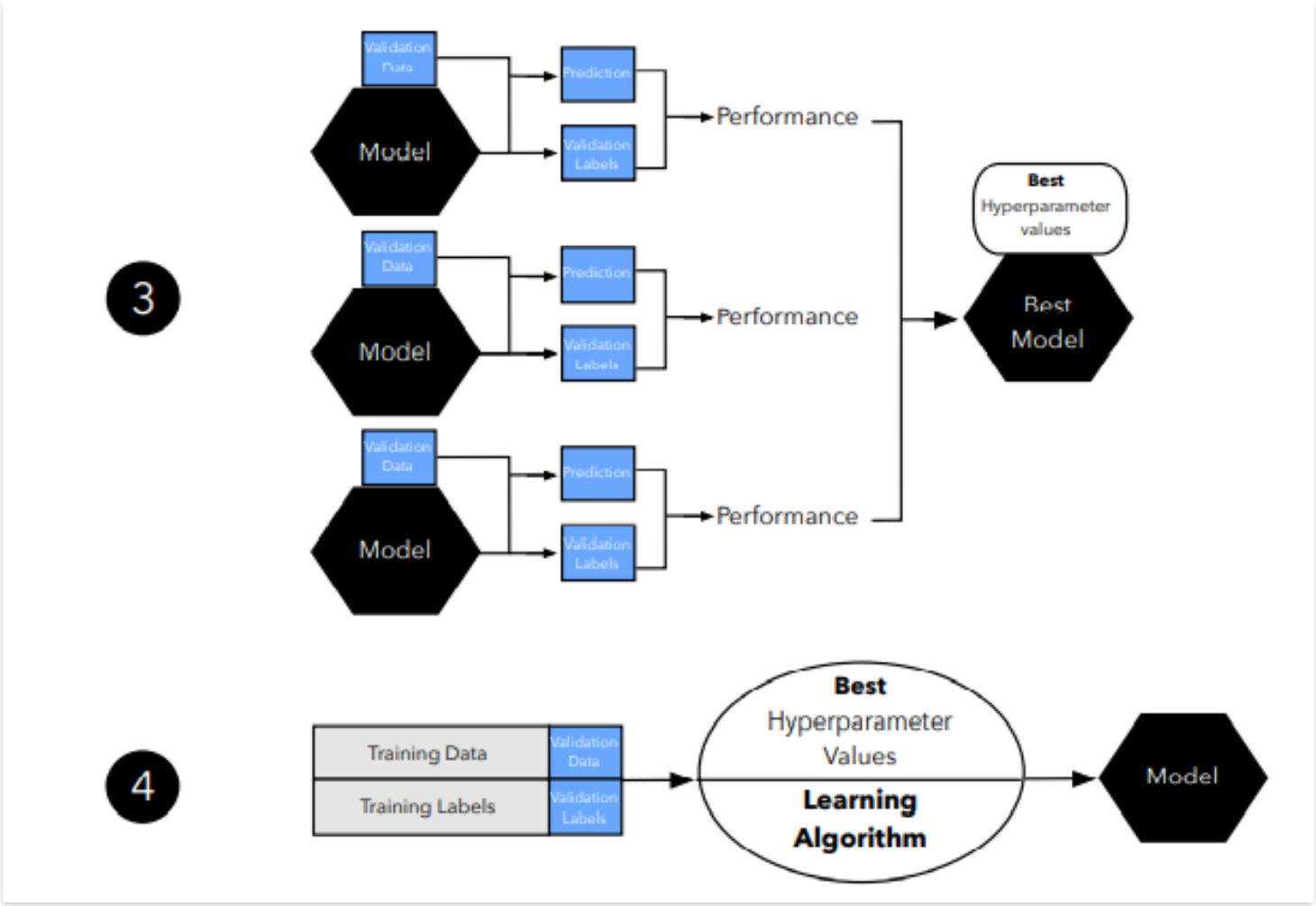
2. Evaluer la performance des différents modèles à l'aide de la base de données de validation.



Three-Way Holdout method

3 Après une comparaison entre les performances des différents modèles on sélectionne les hyperparamètres optimaux.

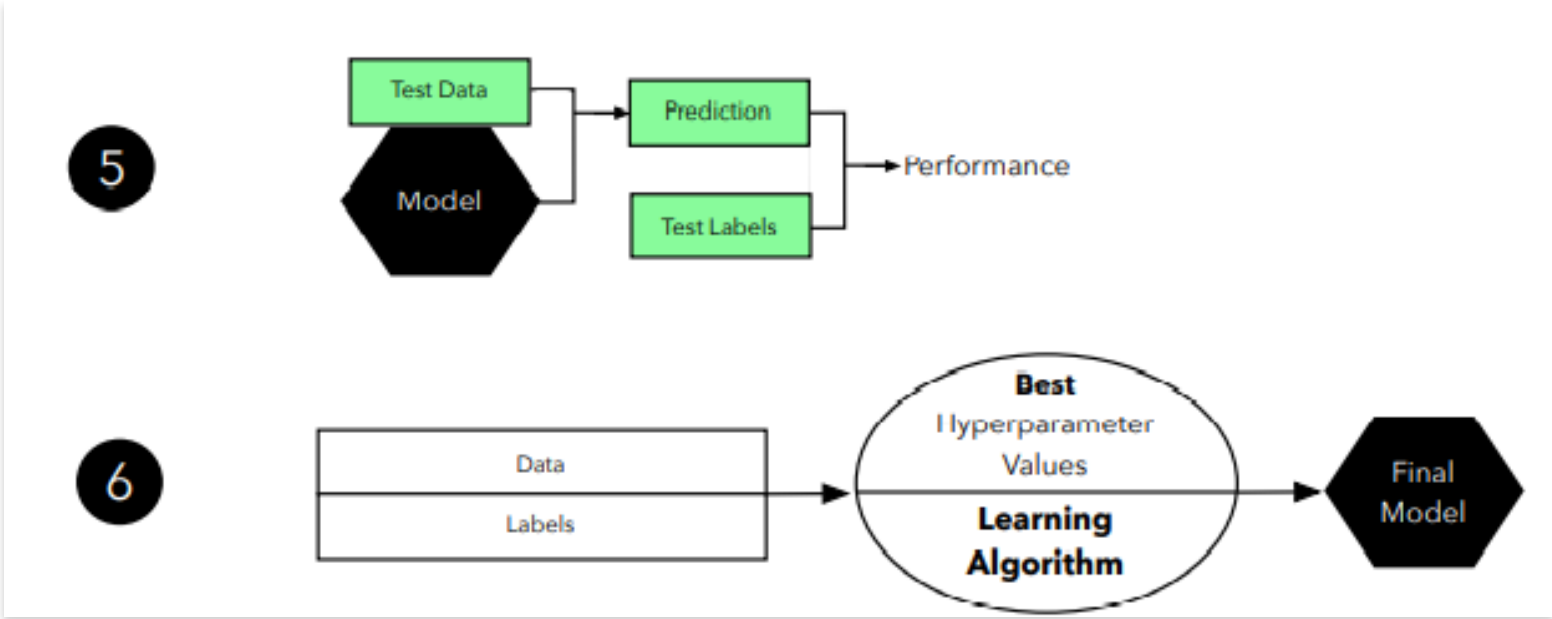
4 Entrainer le modèle à l'aide de la base de données d'apprentissage et la base de données de validation.



Three-Way Holdout method

5 Tester la performance du modèle à l'aide de la BDD test.

6 À ce stade, on peut utiliser la base de données entière pour entrainer le modèle final.

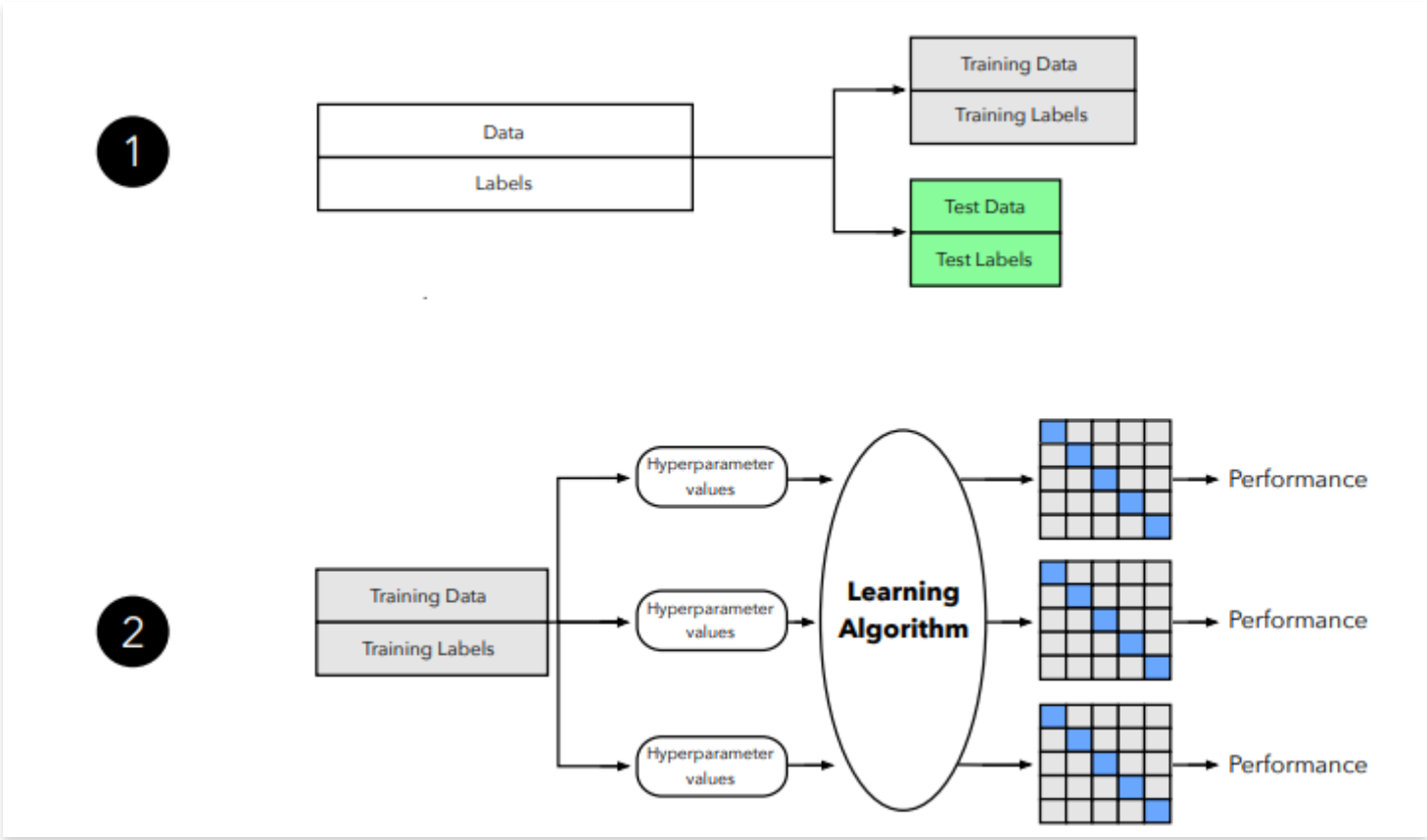


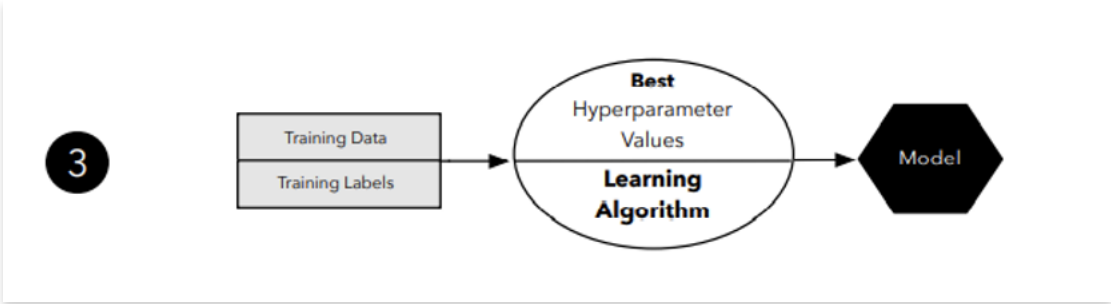
Cross-validation

1. Deviser la base de données initiales en 2 parties, training data et test data.

2. Utiliser la méthode « gridsearch » : pour chaque combinaison des hyperparamètres, utiliser k-fold cross validation sur la base de données d'apprentissage et calculer les estimateurs de performance.

3. Sélectionner les hyperparamètres qui donnent les meilleurs résultats dans la « cross validation » et les utiliser pour entrainer le modèle avec la base de données d'apprentissage entière.





Cross-validation

4. Utiliser la base de données test pour tester la performance du modèle.

5. Si on est satisfait de la validation du modèle, on peut entrainer le modèle à l'aide de la base de données entière (training et testing data) : c'est le modèle à déployer.

